

Figure 10.9
Reason's Swiss cheese model of system failure

In this model, the defenses built into a system are compared to slices of Swiss cheese. Some types of Swiss cheese, such as Emmental, have holes and so the analogy is that the latent conditions are comparable to the holes in cheese slices. The position of these holes is not static but changes depending on the state of the overall sociotechnical system. If each slice represents a barrier, failures can occur when the holes line up at the same time as a human operational error. An active failure of system operation gets through the holes and leads to an overall system failure.

Normally, of course, the holes should not be aligned so operational failures are trapped by the system. To reduce the probability that system failure will result from human error, designers should:

1. Design a system so that different types of barriers are included. This means that the 'holes' will probably be in different places and so there is less chance of the holes lining up and failing to trap an error.
2. Minimize the number of latent conditions in a system. Effectively, this means reducing the number and size of system 'holes'.

Of course, the design of the system as a whole should also attempt to avoid the active failures that can trigger a system failure. This may involve designing the operational processes and the system to ensure that operators are not overworked, distracted, or presented with excessive amounts of information.

10.5.2 System evolution

Large, complex systems have a very long lifetime. During their life, they are changed to correct errors in the original system requirements and to implement new requirements that have emerged. The system's computers are likely to be replaced with new, faster machines. The organization that uses the system may reorganize itself and hence use the system in a different way. The external environment of the system may change, forcing changes to the system. Hence evolution, where the system changes to accommodate environmental change, is a process that runs alongside normal system operational processes. System evolution involves reentering the development process to make changes and extensions to the system's hardware, software, and operational processes.



Legacy systems

Legacy systems are sociotechnical computer-based systems that have been developed in the past, often using older or obsolete technology. These systems include not only hardware and software but also legacy processes and procedures—old ways of doing things that are difficult to change because they rely on legacy software. Changes to one part of the system inevitably involve changes to other components. Legacy systems are often business-critical systems. They are maintained because it is too risky to replace them.

http://www.SoftwareEngineering-9.com/LegacySys/

System evolution, like software evolution (discussed in Chapter 9), is inherently costly for several reasons:

1. Proposed changes have to be analyzed very carefully from a business and a technical perspective. Changes have to contribute to the goals of the system and should not simply be technically motivated.
2. Because subsystems are never completely independent, changes to one subsystem may adversely affect the performance or behavior of other subsystems. Consequent changes to these subsystems may therefore be needed.
3. The reasons for original design decisions are often unrecorded. Those responsible for the system evolution have to work out why particular design decisions were made.
4. As systems age, their structure typically becomes corrupted by change so the costs of making further changes increases.

Systems that have evolved over time are often reliant on obsolete hardware and software technology. If they have a critical role in an organization, they are known as 'legacy systems'. These are usually systems that the organization would like to replace but don't do so as the risks or costs of replacement cannot be justified.

From a dependability and security perspective, changes to a system are often a source of problems and vulnerabilities. If the people implementing the change are different from those who developed the system, they may be unaware that a design decision was made for dependability and security reasons. Therefore, they may change the system and lose some safeguards that were deliberately implemented when the system was built. Furthermore, as testing is so expensive, complete retesting may be impossible after every system change. Adverse side effects of changes that introduce or expose faults in other system components may not then be discovered.

KEY POINTS

- Sociotechnical systems include computer hardware, software, and people, and are situated within an organization. They are designed to support organizational or business goals and objectives.
- Human and organizational factors such as organizational structure and politics have a significant effect on the operation of sociotechnical systems.
- The emergent properties of a system are characteristics of the system as a whole rather than of its component parts. They include properties such as performance, reliability, usability, safety, and security. The success or failure of a system is often dependent on these emergent properties.
- The fundamental systems engineering processes are system procurement, system development, and system operation.
- System procurement covers all of the activities involved in deciding what system to buy and who should supply that system. High-level requirements are developed as part of the procurement process.
- System development includes requirements specification, design, construction, integration, and testing. System integration, where subsystems from more than one supplier must be made to work together, is particularly critical.
- When a system is put into use, the operational processes and the system itself have to change to reflect changing business requirements.
- Human errors are inevitable and systems should include barriers to detect these errors before they lead to system failure. Reason's Swiss cheese model explains how human error plus latent defects in the barriers can lead to system failure.

FURTHER READING

'Airport 95: Automated baggage system'. An excellent, readable case study of what can go wrong with a systems engineering project and how software tends to get the blame for wider systems failures. (*ACM Software Engineering Notes*, 21, March 1996.) http://doi.acm.org/10.1145/227531.227544.

'Software system engineering: A tutorial'. A good general overview of systems engineering, although Thayer focuses exclusively on computer-based systems and does not discuss sociotechnical issues. (R. H. Thayer. *IEEE Computer*, April 2002.) http://dx.doi.org/10.1109/MC.2002.993773.

Trust in Technology: A Socio-technical Perspective. This book is a set of papers that are all concerned, in some way, with the dependability of sociotechnical systems. (K. Clarke, G. Hardstone, M. Rouncefield and I. Sommerville (eds.), Springer, 2006.)

'Fundamentals of Systems Engineering'. This is the introductory chapter in NASA's systems engineering handbook. It presents an overview of the systems engineering process for space

systems. Although these are mostly technical systems, there are sociotechnical issues to be considered. Dependability is obviously critically important. (In *NASA Systems Engineering Handbook*, NASA-SP2007-6105, 2007.) http://education.ksc.nasa.gov/esmdspacegrant/Documents/NASA%20SP-2007-6105%20Rev%201%20Final%2031Dec2007.pdf.

EXERCISES

10.1. Give two examples of government functions that are supported by complex sociotechnical systems and explain why, in the foreseeable future, these functions cannot be completely automated.

10.2. Explain why the environment in which a computer-based system is installed may have unanticipated effects on the system that lead to system failure. Illustrate your answer with a different example from that used in this chapter.

10.3. Why is it impossible to infer the emergent properties of a complex system from the properties of the system components?

10.4. Why is it sometimes difficult to decide whether or not there has been a failure in a sociotechnical system? Illustrate your answer by using examples from the MHC-PMS that has been discussed in earlier chapters.

10.5. What is a 'wicked problem'? Explain why the development of a national medical records system should be considered a 'wicked problem'.

10.6. A multimedia virtual museum system offering virtual experiences of ancient Greece is to be developed for a consortium of European museums. The system should provide users with the facility to view 3-D models of ancient Greece through a standard web browser and should also support an immersive virtual reality experience. What political and organizational difficulties might arise when the system is installed in the museums that make up the consortium?

10.7. Why is system integration a particularly critical part of the systems development process? Suggest three sociotechnical issues that may cause difficulties in the system integration process.

10.8. Explain why legacy systems may be critical to the operation of a business.

10.9. What are the arguments for and against considering system engineering as a profession in its own right, like electrical engineering or software engineering?

10.10. You are an engineer involved in the development of a financial system. During installation, you discover that this system will make a significant number of people redundant. The people in the environment deny you access to essential information to complete the system installation. To what extent should you, as a systems engineer, become involved in this situation? Is it your professional responsibility to complete the installation as contracted? Should you simply abandon the work until the procuring organization has sorted out the problem?

REFERENCES

Ackroyd, S., Harper, R., Hughes, J. A. and Shapiro, D. (1992). *Information Technology and Practical Police Work*. Milton Keynes: Open University Press.

Anderson, R. J., Hughes, J. A. and Sharrock, W. W. (1989). *Working for Profit: The Social Organization of Calculability in an Entrepreneurial Firm*. Aldershot: Avebury.

Checkland, P. (1981). *Systems Thinking, Systems Practice*. Chichester: John Wiley & Sons.

Checkland, P. and Scholes, J. (1990). *Soft Systems Methodology in Action*. Chichester: John Wiley & Sons.

Mumford, E. (1989). 'User Participation in a Changing Environment—Why we need it'. In *Participation in Systems Development*. Knight, K. (ed.). London: Kogan Page.

Reason, J. (2000). 'Human error: Models and management'. *British Medical J.*, **320** 768–70.

Rittel, H. and Webber, M. (1973). 'Dilemmas in a General Theory of Planning'. *Policy Sciences*, **4**, 155–69.

Stevens, R., Brook, P., Jackson, K. and Arnold, S. (1998). *Systems Engineering: Coping with Complexity*. London: Prentice Hall.

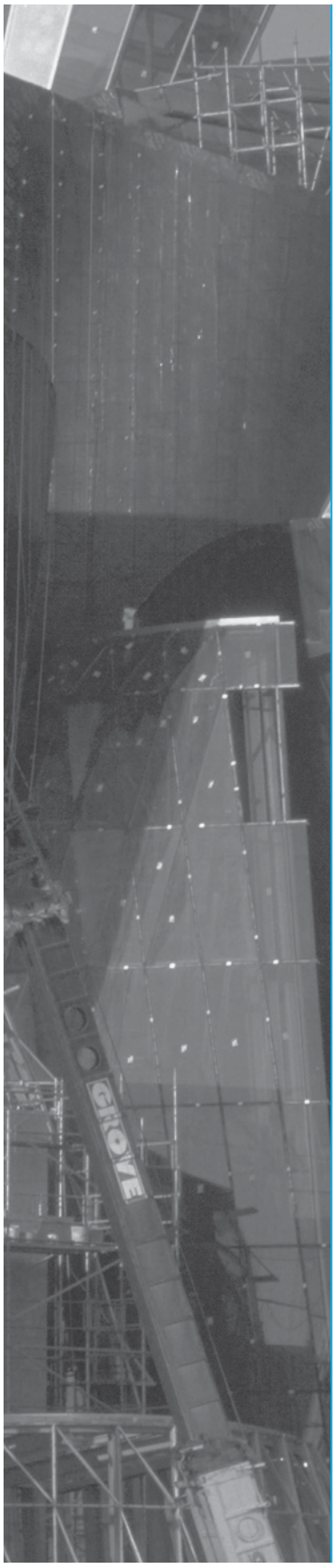
Suchman, L. (1987). *Plans and situated actions: the problem of human-machine communication*. New York: Cambridge University Press.

Swartz, A. J. (1996). 'Airport 95: Automated Baggage System?' *ACM Software Engineering Notes*, **21** (2), 79–83.

Thayer, R. H. (2002). 'Software System Engineering: A Tutorial.' *IEEE Computer*, **35** (4), 68–73.

Thomé, B. (1993). 'Systems Engineering: Principles and Practice of Computer-based Systems Engineering'. Chichester: John Wiley & Sons.

White, S., Alford, M., Holtzman, J., Kuehl, S., McCay, B., Oliver, D., Owens, D., Tully, C. and Willey, A. (1993). 'Systems Engineering of Computer-Based Systems'. *IEEE Computer*, **26** (11), 54–65.



11

Dependability and security

Objectives

The objective of this chapter is to introduce software dependability and security. When you have read this chapter, you will:

- understand why dependability and security are usually more important than the functional characteristics of a software system;
- understand the four principal dimensions of dependability, namely availability, reliability, safety, and security;
- be aware of the specialized terminology that is used when discussing security and dependability;
- understand that to achieve secure, dependable software, you need to avoid mistakes during the development of a system, to detect and remove errors when the system is in use, and to limit the damage caused by operational failures.

Contents

11.1 Dependability properties
11.2 Availability and reliability
11.3 Safety
11.4 Security

As computer systems have become deeply embedded in our business and personal lives, the problems that result from system and software failure are increasing. A failure of server software in an e-commerce company could lead to a major loss of revenue, and possibly also customers for that company. A software error in an embedded control system in a car could lead to expensive recalls of that model for repair and, in the worst case, could be a contributory factor in accidents. The infection of company PCs with malware requires expensive cleanup operations to sort out the problem and could result in the loss or damage to sensitive information.

Because software-intensive systems are so important to governments, companies, and individuals, it is essential that widely used software is trustworthy. The software should be available when required and should operate correctly and without undesirable side effects, such as unauthorized information disclosure. The term 'dependability' was proposed by Laprie (1995) to cover the related systems attributes of availability, reliability, safety, and security. As I discuss in Section 11.1, these properties are inextricably linked, so having a single term to cover them all makes sense.

The dependability of systems is now usually more important than their detailed functionality for the following reasons:

1. *System failures affect a large number of people.* Many systems include functionality that is rarely used. If this functionality were left out of the system, only a small number of users would be affected. System failures, which affect the availability of a system, potentially affect all users of the system. Failure may mean that normal business is impossible.
2. *Users often reject systems that are unreliable, unsafe, or insecure.* If users find that a system is unreliable or insecure, they will refuse to use it. Furthermore, they may also refuse to buy or use other products from the same company that produced the unreliable system, because they believe that these products are also likely to be unreliable or insecure.
3. *System failure costs may be enormous.* For some applications, such as a reactor control system or an aircraft navigation system, the cost of system failure is orders of magnitude greater than the cost of the control system.
4. *Undependable systems may cause information loss.* Data is very expensive to collect and maintain; it is usually worth much more than the computer system on which it is processed. The cost of recovering lost or corrupt data is usually very high.

As I discussed in Chapter 10, software is always part of a broader system. It executes in an operational environment that includes the hardware on which the software executes, the human users of that software, and the organizational or business processes where the software is used. When designing a dependable system, you therefore have to consider:

1. *Hardware failure* System hardware may fail because of mistakes in its design, because components fail as a result of manufacturing errors, or because the components have reached the end of their natural life.



Critical systems

Some classes of system are 'critical systems' where system failure may result in injury to people, damage to the environment, or extensive economic losses. Examples of critical systems include embedded systems in medical devices, such as an insulin pump (safety-critical), spacecraft navigation systems (mission-critical), and online money transfer systems (business critical).

Critical systems are very expensive to develop. Not only must they be developed so that failures are very rare but they must also include recovery mechanisms that are used if and when failures occur.

http://www.SoftwareEngineering-9.com/Web/Dependability/CritSys.html

2. *Software failure* System software may fail because of mistakes in its specification, design, or implementation.
3. *Operational failure* Human users may fail to use or operate the system correctly. As hardware and software have become more reliable, failures in operation are now, perhaps, the largest single cause of system failures.

These failures are often interrelated. A failed hardware component may mean system operators have to cope with an unexpected situation and additional workload. This puts them under stress and people under stress often make mistakes. This can cause the software to fail, which means more work for the operators, even more stress, and so on.

As a result, it is particularly important that designers of dependable, software-intensive systems take a holistic systems perspective, rather than focus on a single aspect of the system such as its software or hardware. If hardware, software, and operational processes are designed separately, without taking into account the potential weaknesses of other parts of the system, then it is more likely that errors will occur at the interfaces between the different parts of the system.

11.1 Dependability properties

All of us are familiar with the problem of computer system failure. For no obvious reason, our computers sometimes crash or go wrong in some way. Programs running on these computers may not operate as expected and occasionally may corrupt the data that is managed by the system. We have learned to live with these failures but few of us completely trust the personal computers that we normally use.

The dependability of a computer system is a property of the system that reflects its trustworthiness. Trustworthiness here essentially means the degree of confidence a user has that the system will operate as they expect, and that the system will not 'fail' in normal use. It is not meaningful to express dependability numerically.

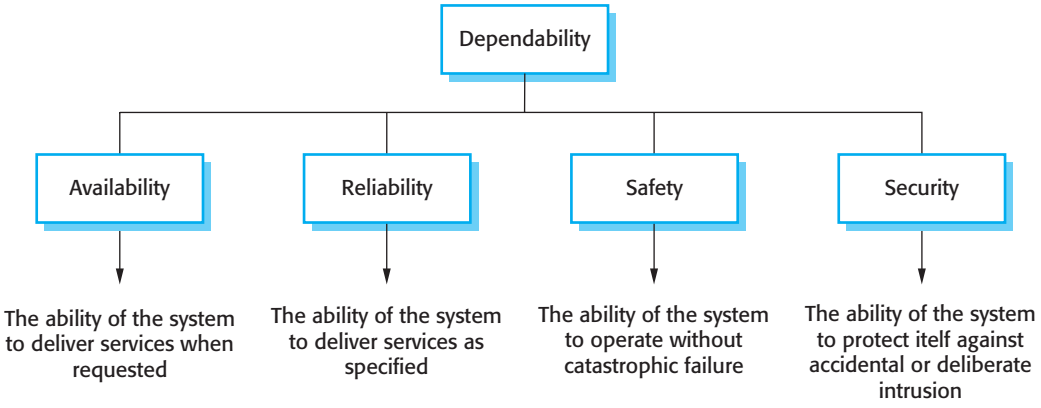


Figure 11.1 Principal dependability properties

Rather, we use relative terms such as 'not dependable,' 'very dependable,' and 'ultra-dependable' to reflect the degrees of trust that we might have in a system.

Trustworthiness and usefulness are not, of course, the same thing. I don't think that the word processor that I used to write this book is a very dependable system. It sometimes freezes and has to be restarted. Nevertheless, because it is very useful, I am prepared to tolerate occasional failure. However, to reflect my lack of trust in the system I save my work frequently and keep multiple backup copies of it. I compensate for the lack of system dependability by actions that limit the damage that could result from system failure.

There are four principal dimensions to dependability, as shown in Figure 11.1.

1. *Availability* Informally, the availability of a system is the probability that it will be up and running and able to deliver useful services to users at any given time.
2. *Reliability* Informally, the reliability of a system is the probability, over a given period of time, that the system will correctly deliver services as expected by the user.
3. *Safety* Informally, the safety of a system is a judgment of how likely it is that the system will cause damage to people or its environment.
4. *Security* Informally, the security of a system is a judgment of how likely it is that the system can resist accidental or deliberate intrusions.

The dependability properties shown in Figure 11.1 are complex properties that can be broken down into a number of other, simpler properties. For example, security includes 'integrity' (ensuring that the systems program and data are not damaged) and 'confidentiality' (ensuring that information can only be accessed by people who are authorized). Reliability includes 'correctness' (ensuring the system services are as specified), 'precision' (ensuring information is delivered at an appropriate level of detail), and 'timeliness' (ensuring that information is delivered when it is required).

Of course, these dependability properties are not all applicable to all systems. For the insulin pump system, introduced in Chapter 1, the most important properties are availability (it must work when required), reliability (it must deliver the correct dose of insulin), and safety (it must never deliver a dangerous dose of insulin). Security is not an issue as the pump will not maintain confidential information. It is not networked and so cannot be maliciously attacked. For the wilderness weather system, availability and reliability are the most important properties because the costs of repair may be very high. For the patient information system, security is particularly important because of the sensitive private data that is maintained.

As well as these four main dependability properties, you may also think of other system properties as dependability properties:

1. *Repairability* System failures are inevitable, but the disruption caused by failure can be minimized if the system can be repaired quickly. For that to happen, it must be possible to diagnose the problem, access the component that has failed, and make changes to fix that component. Repairability in software is enhanced when the organization using the system has access to the source code and has the skills to make changes to it. Open source software makes this easier but the reuse of components can make it more difficult.
2. *Maintainability* As systems are used, new requirements emerge and it is important to maintain the usefulness of a system by changing it to accommodate these new requirements. Maintainable software is software that can be adapted economically to cope with new requirements, and where there is a low probability that making changes will introduce new errors into the system.
3. *Survivability* A very important attribute for Internet-based systems is survivability (Ellison et al., 1999b). Survivability is the ability of a system to continue to deliver service whilst under attack and, potentially, whilst part of the system is disabled. Work on survivability focuses on identifying key system components and ensuring that they can deliver a minimal service. Three strategies are used to enhance survivability—resistance to attack, attack recognition, and recovery from the damage caused by an attack (Ellison et al., 1999a; Ellison et al., 2002). I discuss this in more detail in Chapter 14.
4. *Error tolerance* This property can be considered as part of usability and reflects the extent to which the system has been designed so that user input errors are avoided and tolerated. When user errors occur, the system should, as far as possible, detect these errors and either fix them automatically or request the user to reinput their data.

The notion of system dependability as an encompassing property was introduced because the dependability properties of availability, security, reliability, and safety are closely related. Safe system operation usually depends on the system being available and operating reliably. A system may become unreliable because an intruder has corrupted its data. Denial of service attacks on a system are intended to compromise the

system's availability. If a system is infected with a virus, you cannot then be confident in its reliability or safety because the virus may change its behavior.

To develop dependable software, you therefore need to ensure that:

1. You avoid the introduction of accidental errors into the system during software specification and development.
2. You design verification and validation processes that are effective in discovering residual errors that affect the dependability of the system.
3. You design protection mechanisms that guard against external attacks that can compromise the availability or security of the system.
4. You configure the deployed system and its supporting software correctly for its operating environment.

In addition, you should usually assume that your software is not perfect and that software failures may occur. Your system should therefore include recovery mechanisms that make it possible to restore normal system service as quickly as possible.

The need for fault tolerance means that dependable systems have to include redundant code to help them monitor themselves, detect erroneous states, and recover from faults before failures occur. This affects the performance of systems, as additional checking is required each time the system executes. Therefore, designers usually have to trade off performance and dependability. You may need to leave checks out of the system because these slow the system down. However, the consequential risk here is that some failures occur because a fault has not been detected.

Because of extra design, implementation, and validation costs, increasing the dependability of a system significantly increases development costs. In particular, validation costs are high for systems that must be ultra-dependable such as safety-critical control systems. As well as validating that the system meets its requirements, the validation process may have to prove to an external regulator that the system is safe. For example, aircraft systems have to demonstrate to regulators, such as the Federal Aviation Authority, that the probability of a catastrophic system failure that affects aircraft safety is extremely low.

Figure 11.2 shows that the relationship between costs and incremental improvements in dependability. If your software is not very dependable, you can get significant improvements at relatively low costs by using better software engineering. However, if you are already using good practice, the costs of improvement are much greater and the benefits from that improvement are less. There is also the problem of testing your software to demonstrate that it is dependable. This relies on running many tests and looking at the number of failures that occur. As your software becomes more dependable, you see fewer and fewer failures. Consequently, more and more tests are needed to try and assess how many problems remain in the software. As testing is very expensive, this dramatically increases the cost of high-dependability systems.

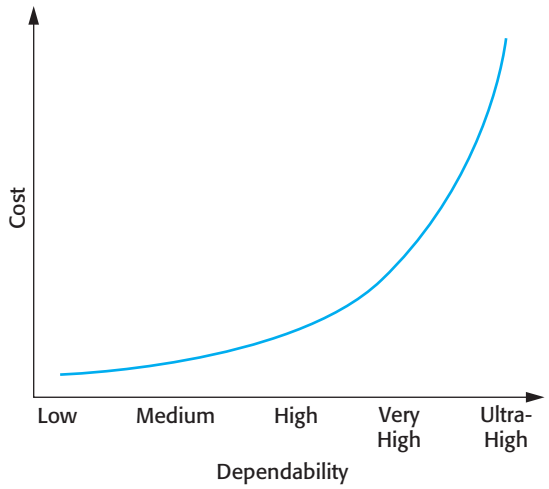


Figure 11.2 Cost/dependability curve

11.2 Availability and reliability

System availability and reliability are closely related properties that can both be expressed as numerical probabilities. The availability of a system is the probability that the system will be up and running to deliver these services to users on request. The reliability of a system is the probability that the system's services will be delivered as defined in the system specification. If, on average, 2 inputs in every 1,000 cause failures, then the reliability, expressed as a rate of occurrence of failure, is 0.002. If the availability is 0.999, this means that, over some time period, the system is available for 99.9% of that time.

Reliability and availability are closely related but sometimes one is more important than the other. If users expect continuous service from a system then the system has a high availability requirement. It must be available whenever a demand is made. However, if the losses that result from a system failure are low and the system can recover quickly then failures don't seriously affect system users. In such systems, the reliability requirements may be relatively low.

A telephone exchange switch that routes phone calls is an example of a system where availability is more important than reliability. Users expect a dial tone when they pick up a phone, so the system has high availability requirements. If a system fault occurs while a connection is being set up, this is often quickly recoverable. Exchange switches can usually reset the system and retry the connection attempt. This can be done very quickly and phone users may not even notice that a failure has occurred. Furthermore, even if a call is interrupted, the consequences are usually not serious. Therefore, availability rather than reliability is the key dependability requirement for this type of system.

System reliability and availability may be defined more precisely as follows:

1. *Reliability* The probability of failure-free operation over a specified time, in a given environment, for a specific purpose.

2. *Availability* The probability that a system, at a point in time, will be operational and able to deliver the requested services.

One of the practical problems in developing reliable systems is that our intuitive notions of reliability and availability are sometimes broader than these limited definitions. The definition of reliability states that the environment in which the system is used and the purpose that it is used for must be taken into account. If you measure system reliability in one environment, you can't assume that the reliability will be the same if the system is used in a different way.

For example, let's say that you measure the reliability of a word processor in an office environment where most users are uninterested in the operation of the software. They follow the instructions for its use and do not try to experiment with the system. If you then measure the reliability of the same system in a university environment, then the reliability may be quite different. Here, students may explore the boundaries of the system and use the system in unexpected ways. This may result in system failures that did not occur in the more constrained office environment.

These standard definitions of availability and reliability do not take into account the severity of failure or the consequences of unavailability. People often accept minor system failures but are very concerned about serious failures that have high consequential costs. For example, computer failures that corrupt stored data are less acceptable than failures that freeze the machine and that can be resolved by restarting the computer.

A strict definition of reliability relates the system implementation to its specification. That is, the system is behaving reliably if its behavior is consistent with that defined in the specification. However, a common cause of perceived unreliability is that the system specification does not match the expectations of the system users. Unfortunately, many specifications are incomplete or incorrect and it is left to software engineers to interpret how the system should behave. As they are not domain experts, they may not, therefore, implement the behavior that users expect. It is also true, of course, that users don't read system specifications. They may therefore have unrealistic expectations of the system.

Availability and reliability are obviously linked as system failures may crash the system. However, availability does not just depend on the number of system crashes, but also on the time needed to repair the faults that have caused the failure. Therefore, if system A fails once a year and system B fails once a month then A is clearly more reliable then B. However, assume that system A takes three days to restart after a failure, whereas system B takes 10 minutes to restart. The availability of system B over the year (120 minutes of down time) is much better than that of system A (4,320 minutes of down time).

The disruption caused by unavailable systems is not reflected in the simple availability metric that specifies the percentage of time that the system is available. The time when the system fails is also significant. If a system is unavailable for an hour each day between 3 am and 4 am, this may not affect many users. However, if the same system is unavailable for 10 minutes during the working day, system unavailability will probably have a much greater effect.

Term	Description
Human error or mistake	Human behavior that results in the introduction of faults into a system. For example, in the wilderness weather system, a programmer might decide that the way to compute the time for the next transmission is to add 1 hour to the current time. This works except when the transmission time is between 23.00 and midnight (midnight is 00.00 in the 24-hour clock).
System fault	A characteristic of a software system that can lead to a system error. The fault is the inclusion of the code to add 1 hour to the time of the last transmission, without a check if the time is greater than or equal to 23.00.
System error	An erroneous system state that can lead to system behavior that is unexpected by system users. The value of transmission time is set incorrectly (to 24.XX rather than 00.XX) when the faulty code is executed.
System failure	An event that occurs at some point in time when the system does not deliver a service as expected by its users. No weather data is transmitted because the time is invalid.

Figure 11.3
Reliability terminology

System reliability and availability problems are mostly caused by system failures. Some of these failures are a consequence of specification errors or failures in other related systems such as a communications system. However, many failures are a consequence of erroneous system behavior that derives from faults in the system. When discussing reliability, it is helpful to use precise terminology and distinguish between the terms 'fault,' 'error,' and 'failure.' I have defined these terms in Figure 11.3 and have illustrated each definition with an example from the wilderness weather system.

When an input or a sequence of inputs causes faulty code in a system to be executed, an erroneous state is created that may lead to a software failure. Figure 11.4, derived from Littlewood (1990), shows a software system as a mapping of a set of inputs to a set of outputs. Given an input or input sequence, the program responds by producing a corresponding output. For example, given an input of a URL, a web browser produces an output that is the display of the requested web page.

Most inputs do not lead to system failure. However, some inputs or input combinations, shown in the shaded ellipse I_e in Figure 11.4, cause system failures or erroneous outputs to be generated. The program's reliability depends on the number of system inputs that are members of the set of inputs that lead to an erroneous output. If inputs in the set I_e are executed by frequently used parts of the system, then failures will be frequent. However, if the inputs in I_e are executed by code that is rarely used, then users will hardly ever see failures.

Because each user of a system uses it in different ways, they have different perceptions of its reliability. Faults that affect the reliability of the system for one user may never be revealed under someone else's mode of working (Figure 11.5). In Figure 11.5, the set of erroneous inputs correspond to the ellipse labeled I_e in Figure 11.4. The set of inputs produced by User 2 intersects with this erroneous input set. User 2 will

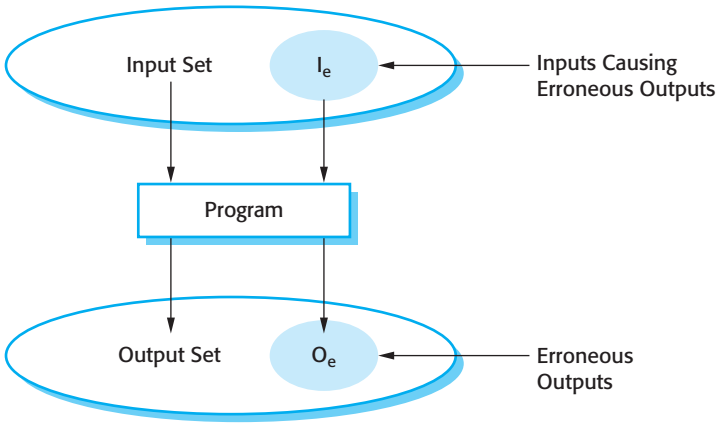


Figure 11.4 A system as an input/output mapping

therefore experience some system failures. User 1 and User 3, however, never use inputs from the erroneous set. For them, the software will always be reliable.

The practical reliability of a program depends on the number of inputs causing erroneous outputs (failures) during normal use of the system by most users. Software faults that only occur in exceptional situations have little practical effect on the system's reliability. Consequently, removing software faults may not significantly improve the overall reliability of the system. Mills et al. (1987) found that removing 60% of known errors in their software led to a 3% reliability improvement. Adams (1984), in a study of IBM software products, noted that many defects in the products were only likely to cause failures after hundreds or thousands of months of product usage.

System faults do not always result in system errors and system errors do not necessarily result in system failures. The reasons for this are as follows:

1. Not all code in a program is executed. The code that includes a fault (e.g., the failure to initialize a variable) may never be executed because of the way that the software is used.

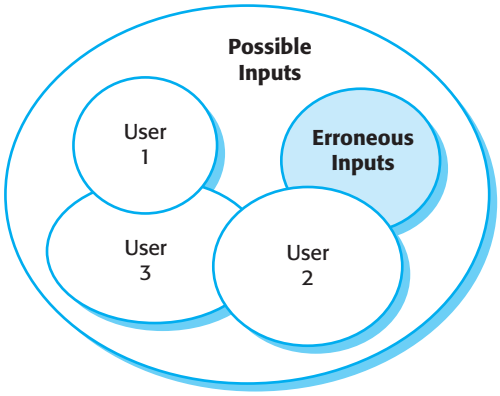


Figure 11.5 Software usage patterns

2. Errors are transient. A state variable may have an incorrect value caused by the execution of faulty code. However, before this is accessed and causes a system failure, some other system input may be processed that resets the state to a valid value.
3. The system may include fault detection and protection mechanisms. These ensure that the erroneous behavior is discovered and corrected before the system services are affected.

Another reason why the faults in a system may not lead to system failures is that, in practice, users adapt their behavior to avoid using inputs that they know cause program failures. Experienced users 'work around' software features that they have found to be unreliable. For example, I avoid certain features, such as automatic numbering in the word processing system that I used to write this book. When I used auto-numbering, it often went wrong. Repairing the faults in unused features makes no practical difference to the system reliability. As users share information on problems and work-arounds, the effects of software problems are reduced.

The distinction between faults, errors, and failures, explained in Figure 11.3, helps identify three complementary approaches that are used to improve the reliability of a system:

1. *Fault avoidance* Development techniques are used that either minimize the possibility of human errors and/or that trap mistakes before they result in the introduction of system faults. Examples of such techniques include avoiding error-prone programming language constructs such as pointers and the use of static analysis to detect program anomalies.
2. *Fault detection and removal* The use of verification and validation techniques that increase the chances that faults will be detected and removed before the system is used. Systematic testing and debugging is an example of a fault-detection technique.
3. *Fault tolerance* These are techniques that ensure that faults in a system do not result in system errors or that system errors do not result in system failures. The incorporation of self-checking facilities in a system and the use of redundant system modules are examples of fault tolerance techniques.

The practical application of these techniques is discussed in Chapter 13, which covers techniques for dependable software engineering.

11.3 Safety

Safety-critical systems are systems where it is essential that system operation is always safe; that is, the system should never damage people or the system's environment even if the system fails. Examples of safety-critical systems include control

and monitoring systems in aircraft, process control systems in chemical and pharmaceutical plants, and automobile control systems.

Hardware control of safety-critical systems is simpler to implement and analyze than software control. However, we now build systems of such complexity that they cannot be controlled by hardware alone. Software control is essential because of the need to manage large numbers of sensors and actuators with complex control laws. For example, advanced, aerodynamically unstable, military aircraft require continual software-controlled adjustment of their flight surfaces to ensure that they do not crash.

Safety-critical software falls into two classes:

1. *Primary safety-critical software* This is software that is embedded as a controller in a system. Malfunctioning of such software can cause a hardware malfunction, which results in human injury or environmental damage. The insulin pump software, introduced in Chapter 1, is an example of a primary safety-critical system. System failure may lead to user injury.
2. *Secondary safety-critical software* This is software that can indirectly result in an injury. An example of such software is a computer-aided engineering design system whose malfunctioning might result in a design fault in the object being designed. This fault may cause injury to people if the designed system malfunctions. Another example of a secondary safety-critical system is the mental health care management system, MHC-PMS. Failure of this system, whereby an unstable patient may not be treated properly, could lead to that patient injuring themselves or others.

System reliability and system safety are related but a reliable system can be unsafe and vice versa. The software may still behave in such a way that the resultant system behavior leads to an accident. There are four reasons why software systems that are reliable are not necessarily safe:

1. We can never be 100% certain that a software system is fault-free and fault-tolerant. Undetected faults can be dormant for a long time and software failures can occur after many years of reliable operation.
2. The specification may be incomplete in that it does not describe the required behavior of the system in some critical situations. A high percentage of system malfunctions (Boehm et al., 1975; Endres, 1975; Lutz, 1993; Nakajo and Kume, 1991) are the result of specification rather than design errors. In a study of errors in embedded systems, Lutz concludes:

   > . . . difficulties with requirements are the key root cause of the safety-related software errors, which have persisted until integration and system testing.

3. Hardware malfunctions may cause the system to behave in an unpredictable way, and present the software with an unanticipated environment. When components are close to physical failure, they may behave erratically and generate signals that are outside the ranges that can be handled by the software.

Term	Definition
Accident (or mishap)	An unplanned event or sequence of events which results in human death or injury, damage to property, or to the environment. An overdose of insulin is an example of an accident.
Hazard	A condition with the potential for causing or contributing to an accident. A failure of the sensor that measures blood glucose is an example of a hazard.
Damage	A measure of the loss resulting from a mishap. Damage can range from many people being killed as a result of an accident to minor injury or property damage. Damage resulting from an overdose of insulin could be serious injury or the death of the user of the insulin pump.
Hazard severity	An assessment of the worst possible damage that could result from a particular hazard. Hazard severity can range from catastrophic, where many people are killed, to minor, where only minor damage results. When an individual death is a possibility, a reasonable assessment of hazard severity is 'very high.'
Hazard probability	The probability of the events occurring which create a hazard. Probability values tend to be arbitrary but range from 'probable' (say 1/100 chance of a hazard occurring) to 'implausible' (no conceivable situations are likely in which the hazard could occur). The probability of a sensor failure in the insulin pump that results in an overdose is probably low.
Risk	This is a measure of the probability that the system will cause an accident. The risk is assessed by considering the hazard probability, the hazard severity, and the probability that the hazard will lead to an accident. The risk of an insulin overdose is probably medium to low.

Figure 11.6
Safety terminology

4. The system operators may generate inputs that are not individually incorrect but which, in some situations, can lead to a system malfunction. An anecdotal example of this occurred when an aircraft undercarriage collapsed whilst the aircraft was on the ground. Apparently, a technician pressed a button that instructed the utility management software to raise the undercarriage. The software carried out the mechanic's instruction perfectly. However, the system should have disallowed the command unless the plane was in the air.

A specialized vocabulary has evolved to discuss safety-critical systems and it is important to understand the specific terms used. Figure 11.6 summarizes some definitions of important terms, with examples taken from the insulin pump system.

The key to assuring safety is to ensure either that accidents do not occur or that the consequences of an accident are minimal. This can be achieved in three complementary ways:

1. *Hazard avoidance* The system is designed so that hazards are avoided. For example, a cutting system that requires an operator to use two hands to press

separate buttons simultaneously avoids the hazard of the operator's hands being in the blade pathway.

2. *Hazard detection and removal* The system is designed so that hazards are detected and removed before they result in an accident. For example, a chemical plant system may detect excessive pressure and open a relief valve to reduce these pressures before an explosion occurs.

3. *Damage limitation* The system may include protection features that minimize the damage that may result from an accident. For example, an aircraft engine normally includes automatic fire extinguishers. If a fire occurs, it can often be controlled before it poses a threat to the aircraft.

Accidents most often occur when several things go wrong at the same time. An analysis of serious accidents (Perrow, 1984) suggests that they were almost all due to a combination of failures in different parts of a system. Unanticipated combinations of subsystem failures led to interactions that resulted in overall system failure. For example, failure of an air-conditioning system could lead to overheating, which then causes the system hardware to generate incorrect signals. Perrow also suggests that it is impossible to anticipate all possible combinations of failures. Accidents are therefore an inevitable part of using complex systems.

Some people have used this as an argument against software control. Because of the complexity of software, there are more interactions between the different parts of a system. This means that there will probably be more combinations of faults that could lead to system failure.

However, software-controlled systems can monitor a wider range of conditions than electro-mechanical systems. They can be adapted relatively easily. They use computer hardware, which has very high inherent reliability and which is physically small and lightweight. Software-controlled systems can provide sophisticated safety interlocks. They can support control strategies that reduce the amount of time people need to spend in hazardous environments. Although software control may introduce more ways in which a system can go wrong, it also allows better monitoring and protection and hence can contribute to improvements in system safety.

In all cases, it is important to maintain a sense of proportion about system safety. It is impossible to make a system 100% safe and society has to decide whether or not the consequences of an occasional accident are worth the benefits that come from the use of advanced technologies. It is also a social and political decision about how to deploy limited national resources to reduce risk to the population as a whole.

11.4 Security

Security is a system attribute that reflects the ability of the system to protect itself from external attacks, which may be accidental or deliberate. These external attacks are possible because most general-purpose computers are now networked and are

Term	Definition
Asset	Something of value which has to be protected. The asset may be the software system itself or data used by that system.
Exposure	Possible loss or harm to a computing system. This can be loss or damage to data, or can be a loss of time and effort if recovery is necessary after a security breach.
Vulnerability	A weakness in a computer-based system that may be exploited to cause loss or harm.
Attack	An exploitation of a system's vulnerability. Generally, this is from outside the system and is a deliberate attempt to cause some damage.
Threats	Circumstances that have potential to cause loss or harm. You can think of these as a system vulnerability that is subjected to an attack.
Control	A protective measure that reduces a system's vulnerability. Encryption is an example of a control that reduces a vulnerability of a weak access control system.

Figure 11.7
Security terminology

therefore accessible by outsiders. Examples of attacks might be the installation of viruses and Trojan horses, unauthorized use of system services or unauthorized modification of a system or its data. If you really want a secure system, it is best not to connect it to the Internet. Then, your security problems are limited to ensuring that authorized users do not abuse the system. In practice, however, there are huge benefits from networked access for most large systems so disconnecting from the Internet is not cost effective.

For some systems, security is the most important dimension of system dependability. Military systems, systems for electronic commerce, and systems that involve the processing and interchange of confidential information must be designed so that they achieve a high level of security. If an airline reservation system is unavailable, for example, this causes inconvenience and some delays in issuing tickets. However, if the system is insecure then an attacker could delete all bookings and it would be practically impossible for normal airline operations to continue.

As with other aspects of dependability, there is a specialized terminology associated with security. Some important terms, as discussed by Pfleeger (Pfleeger and Pfleeger, 2007), are defined in Figure 11.7. Figure 11.8 takes the security concepts described in Figure 11.7 and shows how they relate to the following scenario taken from the MHC-PMS:

> *Clinic staff log on to the MHC-PMS with a username and password. The system requires passwords to be at least eight letters long but allows any password to be set without further checking. A criminal finds out that a well-paid sports star is receiving treatment for mental health problems. He would like to gain illegal access to information in this system so that he can blackmail the star.*

Term	Example
Asset	The records of each patient that is receiving or has received treatment.
Exposure	Potential financial loss from future patients who do not seek treatment because they do not trust the clinic to maintain their data. Financial loss from legal action by the sports star. Loss of reputation.
Vulnerability	A weak password system which makes it easy for users to set guessable passwords. User ids that are the same as names.
Attack	An impersonation of an authorized user.
Threat	An unauthorized user will gain access to the system by guessing the credentials (login name and password) of an authorized user.
Control	A password checking system that disallows user passwords that are proper names or words that are normally included in a dictionary.

Figure 11.8 Examples of security terminology

By posing as a concerned relative and talking with the nurses in the mental health clinic, he discovers how to access the system and personal information about the nurses. By checking name badges, he discovers the names of some of the people allowed access. He then attempts to log on to the system by using these names and systematically guessing possible passwords (such as children's names).

In any networked system, there are three main types of security threats:

1. *Threats to the confidentiality of the system and its data* These can disclose information to people or programs that are not authorized to have access to that information.
2. *Threats to the integrity of the system and its data* These threats can damage or corrupt the software or its data.
3. *Threats to the availability of the system and its data* These threats can restrict access to the software or its data for authorized users.

These threats are, of course, interdependent. If an attack makes the system unavailable, then you will not be able to update information that changes with time. This means that the integrity of the system may be compromised. If an attack succeeds and the integrity of the system is compromised, then it may have to be taken down to repair the problem. Therefore, the availability of the system is reduced.

In practice, most vulnerabilities in sociotechnical systems result from human failings rather than technical problems. People choose easy-to-guess passwords or write

down their passwords in places where they can be found. System administrators make errors in setting up access control or configuration files and users don't install or use protection software. However, as I discussed in Section 10.5, we have to be very careful when classifying a problem as a user error. Human problems often reflect poor systems design decisions that require, for example, frequent password changes (so that users write down their passwords) or complex configuration mechanisms.

The controls that you might put in place to enhance system security are comparable to those for reliability and safety:

1. *Vulnerability avoidance* Controls that are intended to ensure that attacks are unsuccessful. The strategy here is to design the system so that security problems are avoided. For example, sensitive military systems are not connected to public networks so that external access is impossible. You should also think of encryption as a control based on avoidance. Any unauthorized access to encrypted data means that it cannot be read by the attacker. In practice, it is very expensive and time consuming to crack strong encryption.
2. *Attack detection and neutralization* Controls that are intended to detect and repel attacks. These controls involve including functionality in a system that monitors its operation and checks for unusual patterns of activity. If these are detected, then action may be taken, such as shutting down parts of the system, restricting access to certain users, etc.
3. *Exposure limitation and recovery* Controls that support recovery from problems. These can range from automated backup strategies and information 'mirroring' to insurance policies that cover the costs associated with a successful attack on the system.

Without a reasonable level of security, we cannot be confident in a system's availability, reliability, and safety. Methods for certifying availability, reliability, and security assume that the operational software is the same as the software that was originally installed. If the system has been attacked and the software has been compromised in some way (for example, if the software has been modified to include a worm), then the reliability and safety arguments no longer hold.

Errors in the development of a system can lead to security loopholes. If a system does not respond to unexpected inputs or if array bounds are not checked, then attackers can exploit these weaknesses to gain access to the system. Major security incidents such as the original Internet worm (Spafford, 1989) and the Code Red worm more than 10 years later (Berghel, 2001) took advantage of the same vulnerability. Programs in C# do not include array bound checking, so it is possible to overwrite part of memory with code that allows unauthorized access to the system.

KEY POINTS

- Failure of critical computer systems can lead to large economic losses, serious information loss, physical damage, or threats to human life.
- The dependability of a computer system is a system property that reflects the user's degree of trust in the system. The most important dimensions of dependability are availability, reliability, safety, and security.
- The availability of a system is the probability that the system will be able to deliver services to its users when requested to do so. Reliability is the probability that system services will be delivered as specified.
- Perceived reliability is related to the probability of an error occurring in operational use. A program may contain known faults but may still be experienced as reliable by its users. They may never use features of the system that are affected by the faults.
- The safety of a system is a system attribute that reflects the system's ability to operate, normally or abnormally, without injury to people or damage to the environment.
- Security reflects the ability of a system to protect itself against external attacks. Security failures may lead to loss of availability, damage to the system or its data, or the leakage of information to unauthorized people.
- Without a reasonable level of security, the availability, reliability, and safety of the system may be compromised if external attacks damage the system. If a system is unreliable, it is difficult to ensure system safety or security, as they may be compromised by system failures.

FURTHER READING

'The evolution of information assurance'. An excellent article discussing the need to protect critical information in an organization from accidents and attacks. (R. Cummings, *IEEE Computer*, **35** (12), December 2002.) http://dx.doi.org/10.1109/MC.2002.1106181.

'Designing Safety Critical Computer Systems'. This is a good introduction to the field of safety-critical systems, which discusses the fundamental concepts of hazards and risks. More accessible than Dunn's book on safety-critical systems. (W. R. Dunn, *IEEE Computer*, **36** (11), November 2003.) http://dx.doi.org/10.1109/MC.2003.1244533.

Secrets and Lies: Digital Security in a Networked World. An excellent, very readable book on computer security which approaches the subject from a sociotechnical perspective. Schneier's columns on security issues in general (URL below) are also very good. (B. Schneier, John Wiley & Sons, 2004.) http://www.schneier.com/essays.html.

EXERCISES

11.1. Suggest six reasons why software dependability is important in most sociotechnical systems.

11.2. What are the most important dimensions of system dependability?

11.3. Why do the costs of assuring dependability increase exponentially as the reliability requirement increases?

11.4. Giving reasons for your answer, suggest which dependability attributes are likely to be most critical for the following systems:

An Internet server provided by an ISP with thousands of customers

A computer-controlled scalpel used in keyhole surgery

A directional control system used in a satellite launch vehicle

An Internet-based personal finance management system

11.5. Identify six consumer products that are likely to be controlled by safety-critical software systems.

11.6. Reliability and safety are related but distinct dependability attributes. Describe the most important distinction between these attributes and explain why it is possible for a reliable system to be unsafe and vice versa.

11.7. In a medical system that is designed to deliver radiation to treat tumors, suggest one hazard that may arise and propose one software feature that may be used to ensure that the identified hazard does not result in an accident.

11.8. In computer security terms, explain the differences between an attack and a threat.

11.9. Using the MHC-PMS as an example, identify three threats to this system (in addition to the threat shown in Figure 11.8). Suggest controls that might be put in place to reduce the chances of a successful attack based on these threats.

11.10. As an expert in computer security, you have been approached by an organization that campaigns for the rights of torture victims and have been asked to help the organization gain unauthorized access to the computer systems of an American company. This will help them confirm or deny that this company is selling equipment that is used directly in the torture of political prisoners. Discuss the ethical dilemmas that this request raises and how you would react to this request.

REFERENCES

Adams, E. N. (1984). 'Optimizing preventative service of software products'. *IBM J. Res & Dev.*, **28** (1), 2–14.

Berghel, H. (2001). 'The Code Red Worm'. *Comm. ACM*, **44** (12), 15–19.

Boehm, B. W., McClean, R. L. and Urfig, D. B. (1975). 'Some experience with automated aids to the design of large-scale reliable software'. *IEEE Trans. on Software Engineering.*, **SE-1** (1), 125–33.

Ellison, R., Linger, R., Lipson, H., Mead, N. and Moore, A. (2002). 'Foundations of Survivable Systems Engineering'. *Crosstalk: The Journal of Defense Software Engineering*, **12**, 10–15.

Ellison, R. J., Fisher, D. A., Linger, R. C., Lipson, H. F., Longstaff, T. A. and Mead, N. R. (1999a). 'Survivability: Protecting Your Critical Systems'. *IEEE Internet Computing*, **3** (6), 55–63.

Ellison, R. J., Linger, R. C., Longstaff, T. and Mead, N. R. (1999b). 'Survivable Network System Analysis: A Case Study'. *IEEE Software*, **16** (4), 70–7.

Endres, A. (1975). 'An analysis of errors and their causes in system programs'. *IEEE Trans. on Software Engineering.*, **SE-1** (2), 140–9.

Laprie, J.-C. (1995). 'Dependable Computing: Concepts, Limits, Challenges'. FTCS- 25: 25th IEEE Symposium on Fault-Tolerant Computing, Pasadena, Calif.: IEEE Press.

Littlewood, B. (1990). 'Software Reliability Growth Models'. In *Software Reliability Handbook*. Rook, P. (ed.). Amsterdam: Elsevier. 401–412.

Lutz, R. R. (1993). 'Analysing Software Requirements Errors in Safety-Critical Embedded Systems'. RE'93, San Diego, Calif: IEEE.

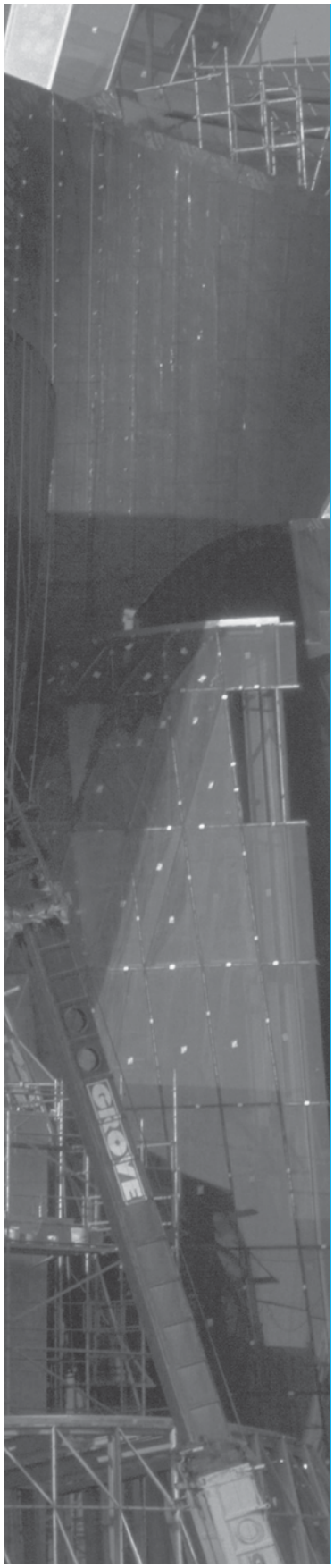
Mills, H. D., Dyer, M. and Linger, R. (1987). 'Cleanroom Software Engineering'. *IEEE Software*, **4** (5), 19–25.

Nakajo, T. and Kume, H. (1991). 'A Case History Analysis of Software Error-Cause Relationships'. *IEEE Trans. on Software Eng.*, **18** (8), 830–8.

Perrow, C. (1984). *Normal Accidents: Living with High-Risk Technology*. New York: Basic Books.

Pfleeger, C. P. and Pfleeger, S. L. (2007). *Security in Computing, 4th edition*. Boston: Addison-Wesley.

Spafford, E. (1989). 'The Internet Worm: Crisis and Aftermath'. *Comm. ACM*, **32** (6), 678–87.



12

Dependability and security specification

Objectives

The objective of this chapter is to explain how to specify functional and non-functional dependability and security requirements. When you have read this chapter, you will:

- understand how a risk-driven approach can be used for identifying and analyzing safety, reliability, and security requirements;
- understand how fault trees can be used to help analyze risks and derive safety requirements;
- have been introduced to metrics for reliability specification and how these are used to specify measurable reliability requirements;
- know the different types of security requirements that may be required in a complex system;
- be aware of the advantages and disadvantages of using formal, mathematical specifications of a system.

Contents

12.1 Risk-driven requirements specification
12.2 Safety specification
12.3 Reliability specification
12.4 Security specification
12.5 Formal specification

In September 1993, a plane landed at Warsaw airport in Poland during a thunderstorm. For nine seconds after landing, the brakes on the computer-controlled braking system did not work. The braking system had not recognized that the plane had landed and assumed that the aircraft was still airborne. A safety feature on the aircraft had stopped the deployment of the reverse thrust system, which slows down the aircraft, because this can be dangerous if the plane is in the air. The plane ran off the end of the runway, hit an earth bank, and caught fire.

The inquiry into the accident showed that the braking system software had operated according to its specification. There were no errors in the program. However, the software specification was incomplete and had not taken into account a rare situation, which arose in this case. The software worked but the system failed.

This illustrates that system dependability does not just depend on good engineering. It also requires attention to detail when the system requirements are derived and the inclusion of special software requirements that are geared to ensuring the dependability and security of a system. Those dependability and security requirements are of two types:

1. Functional requirements, which define checking and recovery facilities that should be included in the system and features that provide protection against system failures and external attacks.
2. Non-functional requirements, which define the required reliability and availability of the system.

The starting point for generating functional dependability and security requirements is often high-level business or domain rules, policies, or regulations. These are high-level requirements that are perhaps best described as ‘shall not’ requirements. By contrast, with normal functional requirements that define what the system shall do, ‘shall not’ requirements define system behavior that is unacceptable. Examples of ‘shall not’ requirements are:

> “The system shall not allow users to modify access permissions on any files that they have not created.” (security)
>
> “The system shall not allow reverse thrust mode to be selected when the aircraft is in flight.” (safety)
>
> “The system shall not allow the simultaneous activation of more than three alarm signals.” (safety)

These ‘shall not’ requirements cannot be implemented directly but have to be decomposed into more specific software functional requirements. Alternatively, they may be implemented through system design decisions such as a decision to use particular types of equipment in the system.

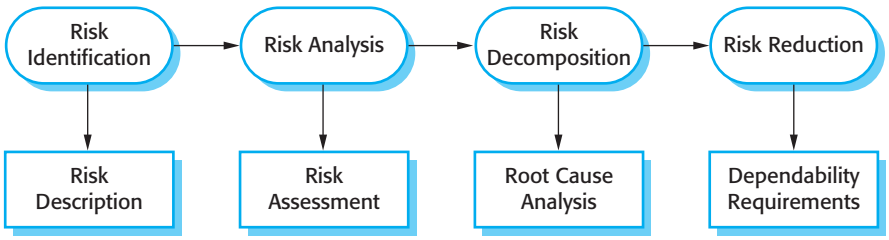


Figure 12.1 Risk-driven specification

12.1 Risk-driven requirements specification

Dependability and security requirements can be thought of as protection requirements. These specify how a system should protect itself from internal faults, stop system failures causing damage to its environment, stop accidents or attacks from the system's environment damaging the system, and facilitate recovery in the event of failure. To discover these protection requirements, you need to understand the risks to the system and its environment. A risk-driven approach to requirements specification takes into account the dangerous events that may occur, the probability that these will actually occur, the probability that damage will result from such an event, and the extent of the damage caused. Security and dependability requirements can then be established, based on an analysis of possible causes of dangerous events.

Risk-driven specification is an approach that has been widely used by safety- and security-critical systems developers. It focuses on those events that could cause the most damage or that are likely to occur frequently. Events that have only minor consequences or that are extremely rare may be ignored. In safety-critical systems, the risks are associated with hazards that can result in accidents; in security-critical systems, the risks come from insider and outsider attacks on a system that are intended to exploit possible vulnerabilities.

A general risk-driven specification process (Figure 12.1) involves understanding the risks faced by the system, discovering their root causes, and generating requirements to manage these risks. The stages in this process are:

1. *Risk identification* Potential risks to the system are identified. These are dependent on the environment in which the system is to be used. Risks may arise from interactions between the system and rare conditions in its operating environment. The Warsaw accident that I discussed earlier happened when crosswinds generated during a thunderstorm caused the plane to tilt so that (unusually) it landed on one wheel rather than two wheels.

2. *Risk analysis and classification* Each risk is considered separately. Those that are potentially serious and not implausible are selected for further analysis.

At this stage, risks may be eliminated because they are unlikely to arise or because they cannot be detected by the software (e.g., an allergic reaction to the sensor in the insulin pump system).

3. *Risk decomposition* Each risk is analyzed to discover potential root causes of that risk. Root causes are the reasons why a system may fail. They may be software or hardware errors or inherent vulnerabilities that result from system design decisions.

4. *Risk reduction* Proposals for ways in which the identified risks may be reduced or eliminated are made. These contribute to the system dependability requirements that define the defenses against the risk and how the risk will be managed.

For large systems, risk analysis may be structured into phases (Leveson, 1995), where each phase considers different types of risks:

1. Preliminary risk analysis, where major risks from the system's environment are identified. These are independent from the technology used for system development. The aim of preliminary risk analysis is to develop an initial set of security and dependability requirements for the system.

2. Life-cycle risk analysis, which takes place during system development and which is mostly concerned with risks that arise from system design decisions. Different technologies and system architectures have their own associated risks. At this stage, you should extend the requirements to protect against these risks.

3. Operational risk analysis, which is concerned with the system user interface and risks from operator errors. Again, once decisions have been made on the user interface design, further protection requirements may have to be added.

These phases are necessary because it is impossible to make all dependability and security decisions without complete information about the system implementation. Security and dependability requirements are particularly affected by technology choices and design decisions. System checks may have to be included to ensure that third-party components have operated correctly. Security requirements may have to be modified because they conflict with the security features that are provided by an off-the-shelf system.

For example, a security requirement may be that users should identify themselves to a system using a pass phrase rather than a password. Pass phrases are considered to be more secure than passwords. They are harder for an attacker to guess or to discover using an automated password cracking system. However, if a decision is made to use an existing system that only supports password-based authentication, then this security requirement cannot be supported. It may then be necessary to include additional functionality in the system to compensate for the increased risks of using passwords rather than pass phrases.



The IEC standard for safety management

The IEC (International Electrotechnical Commission) has defined a standard for safety management for protection systems (i.e., systems that are intended to trigger safeguards when some dangerous situation arises). An example of a protection system is a system that automatically stops a train if it goes through a red signal. This standard includes extensive guidance on the process of safety specification.

http://www.SoftwareEngineering-9.com/Web/SafetyLifeCycle/

12.2 Safety specification

Safety-critical systems are systems in which failures may affect the environment of the system and cause injury or death to the people in that environment. The principal concern of safety specification is to identify requirements that will minimize the probability that such system failures will occur. Safety requirements are primarily protection requirements and are not concerned with normal system operation. They may specify that the system should be shut down so that safety is maintained. In deriving safety requirements, you therefore need to find an acceptable balance between safety and functionality and avoid overprotection. There is no point in building a very safe system if it does not operate in a cost-effective way.

Recall from the discussion in Chapter 10 that safety-critical systems use a specialized terminology where a hazard is something that could (but need not) result in death or injury to a person, and a risk is the probability that the system will enter a hazardous state. Therefore safety specification is usually focused on the hazards that may arise in a given situation, and the events that can lead to these hazards.

The activities in the general risk-based specification process, shown in Figure 12.1, map onto the safety specification process as follows:

1. *Risk identification* In safety specification, this is the hazard identification process that identifies hazards that may threaten the system.
2. *Risk analysis* This is a process of hazard assessment to decide which hazards are the most dangerous and/or the most likely to occur. These should be prioritized when deriving safety requirements.
3. *Risk decomposition* This process is concerned with discovering the events that can lead to the occurrence of a hazard. In safety specification, the process is known as hazard analysis.
4. *Risk reduction* This process is based on the outcome of hazard analysis and leads to identification of safety requirements. These may be concerned with ensuring that a hazard does not arise or lead to an accident or that if an accident does occur, the associated damage is minimized.

12.2.1 Hazard identification

In safety-critical systems, the principal risks come from hazards that can lead to an accident. You can tackle the hazard identification problem by considering different types of hazards, such as physical hazards, electrical hazards, biological hazards, radiation hazards, service failure hazards, and so on. Each of these classes can then be analyzed to discover specific hazards that could occur. Possible combinations of hazards that are potentially dangerous must also be identified.

The insulin pump system that I have used as an example in earlier chapters is a safety-critical system, because failure can cause injury or even death to the system user. Accidents that may occur when using this machine include the user suffering from long-term consequences of poor blood sugar control (eye, heart, and kidney problems); cognitive dysfunction as a result of low blood sugar levels; or the occurrence of some other medical conditions, such as an allergic reaction.

Some of the hazards in the insulin pump system are:

- insulin overdose computation (service failure);
- insulin underdose computation (service failure);
- failure of the hardware monitoring system (service failure);
- power failure due to exhausted battery (electrical);
- electrical interference with other medical equipment such as a heart pacemaker (electrical);
- poor sensor and actuator contact caused by incorrect fitting (physical);
- parts of machine breaking off in patient's body (physical);
- infection caused by introduction of machine (biological);
- allergic reaction to the materials or insulin used in the machine (biological).

Experienced engineers, working with domain experts and professional safety advisers, identify hazards from previous experience and from an analysis of the application domain. Group working techniques such as brainstorming may be used, where a group of people exchange ideas. For the insulin pump system, people who may be involved include doctors, medical physicists, and engineers and software designers.

Software-related hazards are normally concerned with failure to deliver a system service, or with the failure of monitoring and protection systems. Monitoring and protection systems are included in a device to detect conditions, such as low battery levels, which could lead to device failure.

12.2.2 Hazard assessment

The hazard assessment process focuses on understanding the probability that a hazard will occur and the consequences if an accident or incident associated with that hazard should occur. You need to make this analysis to understand whether a hazard

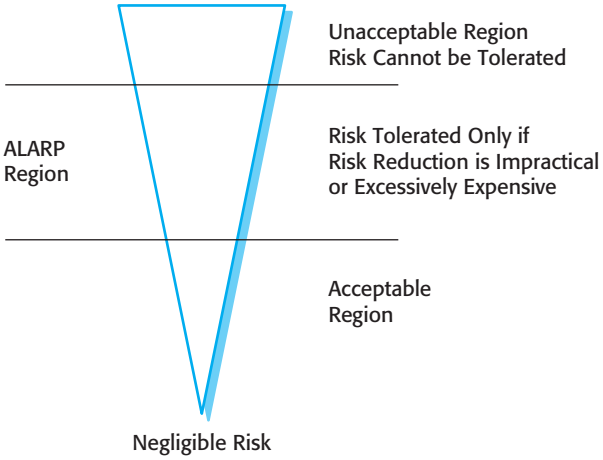


Figure 12.2
The risk triangle

is a serious threat to the system or environment. The analysis also provides a basis for deciding on how to manage the risk associated with the hazard.

For each hazard, the outcome of the analysis and classification process is a statement of acceptability. This is expressed in terms of risk, where the risk takes into account the likelihood of an accident and its consequences. There are three risk categories that you can use in hazard assessment:

1. Intolerable risks in safety-critical systems are those that threaten human life. The system must be designed so that such hazards either cannot arise or, that if they do, features in the system will ensure that they are detected before they cause an accident. In the case of the insulin pump, an intolerable risk is that an overdose of insulin should be delivered.
2. As low as reasonably practical (ALARP) risks are those that have less serious consequences or that are serious but have a very low probability of occurrence. The system should be designed so that the probability of an accident arising because of a hazard is minimized, subject to other considerations such as cost and delivery. An ALARP risk for an insulin pump might be the failure of the hardware monitoring system. The consequences of this are, at worst, a short-term insulin underdose. This is a situation that would not lead to a serious accident.
3. Acceptable risks are those where the associated accidents normally result in minor damage. System designers should take all possible steps to reduce 'acceptable' risks, so long as these do not increase costs, delivery time, or other non-functional system attributes. An acceptable risk in the case of the insulin pump might be the risk of an allergic reaction arising in the user. This usually causes only minor skin irritation. It would not be worth using special, more expensive materials in the device to reduce this risk.

Figure 12.2 (Brazendale and Bell, 1994), developed for safety-critical systems, shows these three regions. The shape of the diagram reflects the costs of ensuring risks do not result in incidents or accidents. The cost of system design to cope with

Identified hazard	Hazard probability	Accident severity	Estimated risk	Acceptability
1. Insulin overdose computation	Medium	High	High	Intolerable
2. Insulin underdose computation	Medium	Low	Low	Acceptable
3. Failure of hardware monitoring system	Medium	Medium	Low	ALARP
4. Power failure	High	Low	Low	Acceptable
5. Machine incorrectly fitted	High	High	High	Intolerable
6. Machine breaks in patient	Low	High	Medium	ALARP
7. Machine causes infection	Medium	Medium	Medium	ALARP
8. Electrical interference	Low	High	Medium	ALARP
9. Allergic reaction	Low	Low	Low	Acceptable

Figure 12.3 Risk classification for the insulin pump

the risk is indicated by the width of the triangle. The highest costs are incurred by risks at the top of the diagram, the lowest costs by risks at the apex of the triangle.

The boundaries between the regions in Figure 12.2 are not technical but rather depend on social and political factors. Over time, society has become more risk-averse so the boundaries have moved downwards. Although the financial costs of accepting risks and paying for any resulting accidents may be less than the costs of accident prevention, public opinion may demand that money be spent to reduce the likelihood of a system accident, thus incurring additional costs.

For example, it may be cheaper for a company to clean up pollution on the rare occasion it occurs, rather than to install systems for pollution prevention. However, because the public and the press will not tolerate such accidents, clearing up the damage rather than preventing the accident is no longer acceptable. Such events may also lead to a reclassification of risk. For example, risks that were thought to be improbable (and hence in the ALARP region) may be reclassified as intolerable because of events, such as terrorist attacks, or accidents that have occurred.

Hazard assessment involves estimating hazard probability and risk severity. This is usually difficult as hazards and accidents are uncommon so the engineers involved may not have direct experience of previous incidents or accidents. Probabilities and severities are assigned using relative terms such as ‘probable,’ ‘unlikely,’ and ‘rare’ and ‘high,’ ‘medium,’ and ‘low’. It is only possible to quantify these terms if enough accident and incident data is available for statistical analysis.

Figure 12.3 shows a risk classification for the hazards identified in the previous section for the insulin delivery system. I have separated the hazards that relate to the

incorrect computation of insulin into an insulin overdose and an insulin underdose. An insulin overdose is potentially more serious than an insulin underdose in the short term. Insulin overdose can result in cognitive dysfunction, coma, and ultimately death. Insulin underdoses lead to high levels of blood sugar. In the short term, these cause tiredness but are not very serious; in the longer term, however, they can lead to serious heart, kidney, and eye problems.

Hazards 4–9 in Figure 12.3 are not software related, but software nevertheless has a role to play in hazard detection. The hardware monitoring software should monitor the system state and warn of potential problems. The warning will often allow the hazard to be detected before it causes an accident. Examples of hazards that might be detected are power failure, which is detected by monitoring the battery, and incorrect placement of machine, which may be detected by monitoring signals from the blood sugar sensor.

The monitoring software in the system is, of course, safety related. Failure to detect a hazard could result in an accident. If the monitoring system fails but the hardware is working correctly then this is not a serious failure. However, if the monitoring system fails and hardware failure cannot then be detected, then this could have more serious consequences.

12.2.3 Hazard analysis

Hazard analysis is the process of discovering the root causes of hazards in a safety-critical system. Your aim is to find out what events or combination of events could cause a system failure that results in a hazard. To do this, you can use either a top-down or a bottom-up approach. Deductive, top-down techniques, which tend to be easier to use, start with the hazard and work up from that to the possible system failure. Inductive, bottom-up techniques start with a proposed system failure and identify what hazards might result from that failure.

Various techniques have been proposed as possible approaches to hazard decomposition or analysis. These are summarized by Storey (1996). They include reviews and checklists, formal techniques such as Petri net analysis (Peterson, 1981), formal logic (Jahanian and Mok, 1986), and fault tree analysis (Leveson and Stolzy, 1987; Storey, 1996). As I don't have space to cover all of these techniques here, I focus on a widely used approach to hazard analysis based on fault trees. This technique is fairly easy to understand without specialist domain knowledge.

To do a fault tree analysis, you start with the hazards that have been identified. For each hazard, you then work backwards to discover the possible causes of that hazard. You put the hazard at the root of the tree and identify the system states that can lead to that hazard. For each of these states, you then identify further system states that can lead to them. You continue this decomposition until you reach the root cause(s) of the risk. Hazards that can only arise from a combination of root causes are usually less likely to lead to an accident than hazards with a single root cause.

Figure 12.4 is a fault tree for the software-related hazards in the insulin delivery system that could lead to an incorrect dose of insulin being delivered. In this case, I have

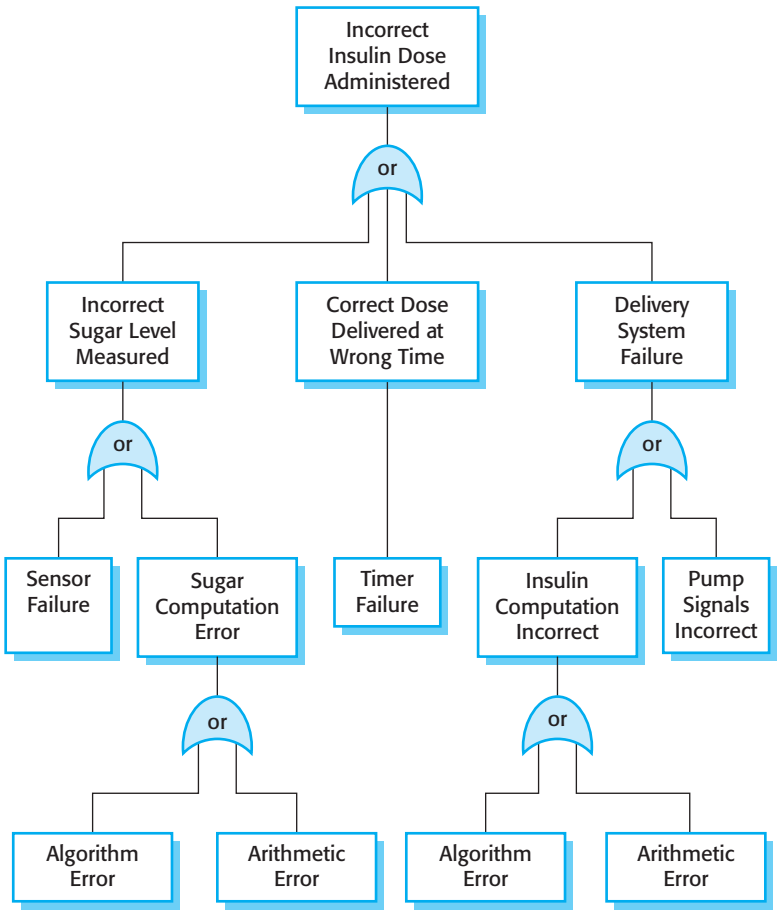


Figure 12.4 An example of a fault tree

merged insulin underdose and insulin overdose into a single hazard, namely 'incorrect insulin dose administered.' This reduces the number of fault trees that are required. Of course, when you specify how the software should react to this hazard, you have to distinguish between an insulin underdose and an insulin overdose. As I have said, they are not equally serious—in the short term, an overdose is the more serious hazard.

From Figure 12.4, you can see that:

1. There are three conditions that could lead to the administration of an incorrect dose of insulin. The level of blood sugar may have been incorrectly measured so the insulin requirement has been computed with an incorrect input. The delivery system may not respond correctly to commands specifying the amount of insulin to be injected. Alternatively, the dose may be correctly computed but it is delivered too early or too late.
2. The left branch of the fault tree, concerned with incorrect measurement of the blood sugar level, looks at how this might happen. This could occur either

because the sensor that provides an input to calculate the sugar level has failed or because the calculation of the blood sugar level has been carried out incorrectly. The sugar level is calculated from some measured parameter, such as the conductivity of the skin. Incorrect computation can result from either an incorrect algorithm or an arithmetic error that results from the use of floating point numbers.

3. The central branch of the tree is concerned with timing problems and concludes that these can only result from system timer failure.
4. The right branch of the tree, concerned with delivery system failure, examines possible causes of this failure. These could result from an incorrect computation of the insulin requirement, or from a failure to send the correct signals to the pump that delivers the insulin. Again, an incorrect computation can result from algorithm failure or arithmetic errors.

Fault trees are also used to identify potential hardware problems. Hardware fault trees may provide insights into requirements for software to detect and, perhaps, correct these problems. For example, insulin doses are not administered at a very high frequency, no more than two or three times per hour and sometimes less often than this. Therefore, processor capacity is available to run diagnostic and self-checking programs. Hardware errors such as sensor, pump, or timer errors can be discovered and warnings issued before they have a serious effect on the patient.

12.2.4 Risk reduction

Once potential risks and their root causes have been identified, you are then able to derive safety requirements that manage the risks and ensure that incidents or accidents do not occur. There are three possible strategies that you can use:

1. *Hazard avoidance* The system is designed so that the hazard cannot occur.
2. *Hazard detection and removal* The system is designed so that hazards are detected and neutralized before they result in an accident.
3. *Damage limitation* The system is designed so that the consequences of an accident are minimized.

Normally, designers of critical systems use a combination of these approaches. In a safety-critical system, intolerable hazards may be handled by minimizing their probability and adding a protection system that provides a safety backup. For example, in a chemical plant control system, the system will attempt to detect and avoid excess pressure in the reactor. However, there may also be an independent protection system that monitors the pressure and opens a relief valve if high pressure is detected.

In the insulin delivery system, a ‘safe state’ is a shutdown state where no insulin is injected. Over a short period this is not a threat to the diabetic’s health. For the

SR1: The system shall not deliver a single dose of insulin that is greater than a specified maximum dose for a system user.
SR2: The system shall not deliver a daily cumulative dose of insulin that is greater than a specified maximum daily dose for a system user.
SR3: The system shall include a hardware diagnostic facility that shall be executed at least four times per hour.
SR4: The system shall include an exception handler for all of the exceptions that are identified in Table 3.
SR5: The audible alarm shall be sounded when any hardware or software anomaly is discovered and a diagnostic message, as defined in Table 4, shall be displayed.
SR6: In the event of an alarm, insulin delivery shall be suspended until the user has reset the system and cleared the alarm.

Figure 12.5 Examples of safety requirements

software failures that could lead to an incorrect dose of insulin are considered, the following 'solutions' might be developed:

1. *Arithmetic error* This may occur when an arithmetic computation causes a representation failure. The specification should identify all possible arithmetic errors that may occur and state that an exception handler must be included for each possible error. The specification should set out the action to be taken for each of these errors. The default safe action is to shut down the delivery system and activate a warning alarm.
2. *Algorithmic error* This is a more difficult situation as there is no clear program exception that must be handled. This type of error could be detected by comparing the required insulin dose computed with the previously delivered dose. If it is much higher, this may mean that the amount has been computed incorrectly. The system may also keep track of the dose sequence. After a number of above-average doses have been delivered, a warning may be issued and further dosage limited.

Some of the resulting safety requirements for the insulin pump software are shown in Figure 12.5. These are user requirements and, naturally, they would be expressed in more detail in the system requirements specification. In Figure 12.5, the references to Tables 3 and 4 relate to tables that are included in the requirements document—they are not shown here.

12.3 Reliability specification

As I discussed in Chapter 10, the overall reliability of a system depends on the hardware reliability, the software reliability, and the reliability of the system operators. The system software has to take this into account. As well as including requirements

that compensate for software failure, there may also be related reliability requirements to help detect and recover from hardware failures and operator errors.

Reliability is different from safety and security in that it is a measurable system attribute. That is, it is possible to specify the level of reliability that is required, monitor the system's operation over time, and check if the required reliability has been achieved. For example, a reliability requirement might be that system failures that require a reboot should not occur more than once per week. Every time such a failure occurs, it can be logged and you can check if the required level of reliability has been achieved. If not, you either modify your reliability requirement or submit a change request to address the underlying system problems. You may decide to accept a lower level of reliability because of the costs of changing the system to improve reliability or because fixing the problem may have adverse side effects, such as lower performance or throughput.

By contrast, both safety and security are about avoiding undesirable situations, rather than specifying a desired 'level' of safety or security. Even one such situation in the lifetime of a system may be unacceptable and, if it occurs, system changes have to be made. It makes no sense to make statements like 'system faults should result in fewer than 10 injuries per year.' As soon as one injury occurs, the system problem must be rectified.

Reliability requirements are, therefore, of two kinds:

1. Non-functional requirements, which define the number of failures that are acceptable during normal use of the system, or the time in which the system is unavailable for use. These are quantitative reliability requirements.
2. Functional requirements, which define system and software functions that avoid, detect, or tolerate faults in the software and so ensure that these faults do not lead to system failure.

Quantitative reliability requirements lead to related functional system requirements. To achieve some required level of reliability, the functional and design requirements of the system should specify the faults to be detected and the actions that should be taken to ensure that these faults do not lead to system failures.

The process of reliability specification can be based on the general risk-driven specification process shown in Figure 12.1:

1. *Risk identification* At this stage, you identify the types of system failures that may lead to economic losses of some kind. For example, an e-commerce system may be unavailable so that customers cannot place orders, or a failure that corrupts data may require time to restore the system database from a backup and rerun transactions that have been processed. The list of possible failure types, shown in Figure 12.6, can be used as a starting point for risk identification.
2. *Risk analysis* This involves estimating the costs and consequences of different types of software failure and selecting high-consequence failures for further analysis.

Failure type	Description
Loss of service	The system is unavailable and cannot deliver its services to users. You may separate this into loss of critical services and loss of non-critical services, where the consequences of a failure in non-critical services are less than the consequences of critical service failure.
Incorrect service delivery	The system does not deliver a service correctly to users. Again, this may be specified in terms of minor and major errors or errors in the delivery of critical and non-critical services.
System/data corruption	The failure of the system causes damage to the system itself or its data. This will usually but not necessarily be in conjunction with other types of failures.

Figure 12.6 Types of system failure

3. *Risk decomposition* At this stage, you do a root cause analysis of serious and probable system failures. However, this may be impossible at the requirements stage as the root causes may depend on system design decisions. You may have to return to this activity during design and development.

4. *Risk reduction* At this stage, you should generate quantitative reliability specifications that set out the acceptable probabilities of the different types of failures. These should, of course, take into account the costs of failures. You may use different probabilities for different system services. You may also generate functional reliability requirements. Again, this may have to wait until system design decisions have been made. However, as I discuss in Section 12.3.2, it is sometimes difficult to create quantitative specifications. You may only be able to identify functional reliability requirements.

12.3.1 Reliability metrics

In general terms, reliability can be specified as a probability that a system failure will occur when a system is in use within a specified operating environment. If you are willing to accept, for example, that 1 in any 1,000 transactions may fail, then you can specify the failure probability as 0.001. This doesn't mean, of course, that you will see 1 failure in every 1,000 transactions. It means that if you observe N thousand transactions, the number of failures that you observe should be around N. You can refine this for different kinds of failure or for different parts of the system. You may decide that critical components must have a lower probability of failure than noncritical components.

There are two important metrics that are used to specify reliability plus an additional metric that is used to specify the related system attribute of availability. The choice of metric depends on the type of system that is being specified and the requirements of the application domain. The metrics are:

1. *Probability of failure on demand (POFOD)* If you use this metric, you define the probability that a demand for service from a system will result in a system

Availability	Explanation
0.9	The system is available for 90% of the time. This means that, in a 24-hour period (1,440 minutes), the system will be unavailable for 144 minutes.
0.99	In a 24-hour period, the system is unavailable for 14.4 minutes.
0.999	The system is unavailable for 84 seconds in a 24-hour period.
0.9999	The system is unavailable for 8.4 seconds in a 24-hour period. Roughly, one minute per week.

Figure 12.7 Availability specification

failure. So, POFOD = 0.001 means that there is a 1/1,000 chance that a failure will occur when a demand is made.

2. *Rate of occurrence of failures (ROCOF)* This metric sets out the probable number of system failures that are likely to be observed relative to a certain time period (e.g., an hour), or to the number of system executions. In the example above, the ROCOF is 1/1,000. The reciprocal of ROCOF is the mean time to failure (MTTF), which is sometimes used as a reliability metric. MTTF is the average number of time units between observed system failures. Therefore, a ROCOF of two failures per hour implies that the mean time to failure is 30 minutes.

3. *Availability (AVAIL)* The availability of a system reflects its ability to deliver services when requested. AVAIL is the probability that a system will be operational when a demand is made for service. Therefore, an availability of 0.9999, means that, on average, the system will be available for 99.99% of the operating time. Figure 12.7 shows what different levels of availability mean in practice.

POFOD should be used as a reliability metric in situations where a failure on demand can lead to a serious system failure. This applies irrespective of the frequency of the demands. For example, a protection system that monitors a chemical reactor and shuts down the reaction if it is overheating should have its reliability specified using POFOD. Generally, demands on a protection system are infrequent as the system is a last line of defense, after all other recovery strategies have failed. Therefore a POFOD of 0.001 (1 failure in 1,000 demands) might seem to be risky, but if there are only two or three demands on the system in its lifetime, then you will probably never see a system failure.

ROCOF is the most appropriate metric to use in situations where demands on systems are made regularly rather than intermittently. For example, in a system that handles a large number of transactions, you may specify a ROCOF of 10 failures per day. This means that you are willing to accept that an average of 10 transactions per day will not complete successfully and will have to be canceled. Alternatively, you may specify ROCOF as the number of failures per 1,000 transactions.

If the absolute time between failures is important, you may specify the reliability as the mean time between failures. For example, if you are specifying the required

reliability for a system with long transactions (such as a computer-aided design system), you should specify the reliability with a long mean time to failure. The MTTF should be much longer than the average time that a user works on his or her models without saving their results. This would mean that users would be unlikely to lose work through a system failure in any one session.

To assess the reliability of a system, you have to capture data about its operation. The data required may include:

1. The number of system failures given a number of requests for system services. This is used to measure the POFOD.
2. The time or the number of transactions between system failures plus the total elapsed time or total number of transactions. This is used to measure ROCOF and MTTF.
3. The repair or restart time after a system failure that leads to loss of service. This is used in the measurement of availability. Availability does not just depend on the time between failures but also on the time required to get the system back into operation.

The time units that may be used are calendar time or processor time or a discrete unit such as number of transactions. In systems that spend much of their time waiting to respond to a service request, such as telephone switching systems, the time unit that should be used is processor time. If you use calendar time, then this will include the time when the system was doing nothing.

You should use calendar time for systems that are in continuous operation. Monitoring systems, such as alarm systems, and other types of process control systems fall into this category. Systems that process transactions such as bank ATMs or airline reservation systems have variable loads placed on them depending on the time of day. In these cases, the unit of 'time' used could be the number of transactions (i.e., the ROCOF would be number of failed transactions per N thousand transactions).

12.3.2 Non-functional reliability requirements

Non-functional reliability requirements are quantitative specifications of the required reliability and availability of a system, calculated using one of the metrics described in the previous section. Quantitative reliability and availability specification has been used for many years in safety-critical systems but is only rarely used in business critical systems. However, as more and more companies demand 24/7 service from their systems, it is likely that such techniques will be increasingly used.

There are several advantages in deriving quantitative reliability specifications:

1. The process of deciding what required level of the reliability helps to clarify what stakeholders really need. It helps stakeholders understand that there are different types of system failure, and it makes clear to them that high levels of reliability are very expensive to achieve.

2. It provides a basis for assessing when to stop testing a system. You stop when the system has achieved its required reliability level.
3. It is a means of assessing different design strategies intended to improve the reliability of a system. You can make a judgment about how each strategy might lead to the required levels of reliability.
4. If a regulator has to approve a system before it goes into service (e.g., all systems that are critical to flight safety on an aircraft are regulated), then evidence that a required reliability target has been met is important for system certification.

To establish the required level of system reliability, you have to consider the associated losses that could result from a system failure. These are not simply financial losses, but also loss of reputation for a business. Loss of reputation means that customers will go elsewhere. Although the short-term losses from a system failure may be relatively small, the longer-term losses may be much more significant. For example, if you try to access an e-commerce site and find that it is unavailable, you may try to find what you want elsewhere rather than wait for the system to become available. If this happens more than once, you will probably not shop at that site again.

The problem with specifying reliability using metrics such as POFOD, ROCOF, and AVAIL is that it is possible to overspecify reliability and thus incur high development and validation costs. The reason for this is that system stakeholders find it difficult to translate their practical experience into quantitative specifications. They may think that a POFOD of 0.001 (1failure in 1,000 demands) represents a relatively unreliable system. However, as I have explained, if demands for a service are uncommon, it actually represents a very high level of reliability.

If you specify reliability as a metric, it is obviously important to assess that the required level of reliability has been achieved. You do this assessment as part of system testing. To assess the reliability of a system statistically, you have to observe a number of failures. If you have, for example, a POFOD of 0.0001 (1 failure in 10,000 demands), then you may have to design tests that make 50 or 60 thousand demands on a system and where several failures are observed. It may be practically impossible to design and implement this number of tests. Therefore, overspecification of reliability leads to very high testing costs.

When you specify the availability of a system, you may have similar problems. Although a very high level of availability may seem to be desirable, most systems have very intermittent demand patterns (e.g., a business system will mostly be used during normal business hours) and a single availability figure does not really reflect user needs. You need high availability when the system is being used but not at other times. Depending, of course, on the type of system, there may be no real practical difference between an availability of 0.999 and an availability of 0.9999.

A fundamental problem with overspecification is that it may be practically impossible to show that a very high level of reliability or availability has been achieved. For example, say a system was intended for use in a safety-critical application and was therefore required to never fail over its total lifetime. Assume that 1,000 copies of the system are to be installed and the system is executed 1,000

times per second. The projected lifetime of the system is 10 years. The total number of system executions is therefore approximately $3*10^{14}$. There is no point in specifying that the rate of occurrence of failure should be $1/10^{15}$ executions (this allows for some safety factor) as you cannot test the system for long enough to validate this level of reliability.

Organizations must therefore be realistic about whether it is worth specifying and validating a very high level of reliability. High reliability levels are clearly justified in systems where reliable operation is critical, such as telephone switching systems, or where system failure may result in large economic losses. They are probably not justified for many types of business or scientific systems. Such systems have modest reliability requirements, as the costs of failure are simply processing delays and it is straightforward and relatively inexpensive to recover from these.

There are a number of steps that you can take to avoid the overspecification of system reliability:

1. Specify the availability and reliability requirements for different types of failures. There should be a lower probability of serious failures occurring than minor failures.
2. Specify the availability and reliability requirements for different services separately. Failures that affect the most critical services should be specified as less probable than those with only local effects. You may decide to limit the quantitative reliability specification to the most critical system services.
3. Decide whether you really need high reliability in a software system or whether the overall system dependability goals can be achieved in other ways. For example, you may use error detection mechanisms to check the outputs of a system and have processes in place to correct errors. There may then be no need for a high level of reliability in the system that generates the outputs.

To illustrate this latter point, consider the reliability requirements for a bank ATM system that dispenses cash and provides other services to customers. If there are hardware or software ATM problems, then these lead to incorrect entries in the customer account database. These could be avoided by specifying a very high level of hardware and software reliability in the ATM.

However, banks have many years of experience of how to identify and correct incorrect account transactions. They use accounting methods to detect when things have gone wrong. Most transactions that fail can simply be canceled, resulting in no loss to the bank and minor customer inconvenience. Banks that run ATM networks therefore accept that ATM failures may mean that a small number of transactions are incorrect but they think it more cost effective to fix these later rather than to incur very high costs in avoiding faulty transactions.

For a bank (and for the bank's customers), the availability of the ATM network is more important than whether or not individual ATM transactions fail. Lack of availability means more demand on counter services, customer dissatisfaction, engineering costs to repair the network, etc. Therefore, for transaction-based systems, such as

banking and e-commerce systems, the focus of reliability specification is usually on specifying the availability of the system.

To specify the availability of an ATM network, you should identify the system services and specify the required availability for each of these. These are:

- the customer account database service;
- the individual services provided by an ATM such as 'withdraw cash,' 'provide account information,' etc.

Here, the database service is most critical as failure of this service means that all of the ATMs in the network are out of action. Therefore, you should specify this to have a high level of availability. In this case, an acceptable figure for database availability (ignoring issues such as scheduled maintenance and upgrades) would probably be around 0.9999, between 7 am and 11 pm. This means a down time of less than one minute per week. In practice, this would mean that very few customers would be affected and would only lead to minor customer inconvenience.

For an individual ATM, the overall availability depends on mechanical reliability and the fact that it can run out of cash. Software issues are likely to have less effect than factors such as these. Therefore, a lower level of availability for the ATM software is acceptable. The overall availability of the ATM software might therefore be specified as 0.999, which means that a machine might be unavailable for between one and two minutes each day.

To illustrate failure-based reliability specification, consider the reliability requirements for the control software in the insulin pump. This system delivers insulin a number of times per day and monitors the user's blood glucose several times per hour. Because the use of the system is intermittent and failure consequences are serious, the most appropriate reliability metric is POFOD (probability of failure on demand).

There are two possible types of failure in the insulin pump:

1. Transient software failures that can be repaired by user actions such as resetting or recalibrating the machine. For these types of failures, a relatively low value of POFOD (say 0.002) may be acceptable. This means that one failure may occur in every 500 demands made on the machine. This is approximately once every 3.5 days, because the blood sugar is checked about five times per hour.
2. Permanent software failures that require the software to be reinstalled by the manufacturer. The probability of this type of failure should be much lower. Roughly once a year is the minimum figure, so POFOD should be no more than 0.00002.

However, failure to deliver insulin does not have immediate safety implications, so commercial factors rather than the safety factors govern the level of reliability required. Service costs are high because users need fast repair and replacement. It is in the manufacturer's interest to limit the number of permanent failures that require repair.

RR1: A pre-defined range for all operator inputs shall be defined and the system shall check that all operator inputs fall within this pre-defined range. (Checking)
RR2: Copies of the patient database shall be maintained on two separate servers that are not housed in the same building. (Recovery, redundancy)
RR3: N-version programming shall be used to implement the braking control system. (Redundancy)
RR4: The system must be implemented in a safe subset of Ada and checked using static analysis. (Process)

Figure 12.8 Examples of functional reliability requirements

12.3.3 Functional reliability specification

Functional reliability specification involves identifying requirements that define constraints and features that contribute to system reliability. For systems where the reliability has been quantitatively specified, these functional requirements may be necessary to ensure that a required level of reliability is achieved.

There are three types of functional reliability requirements for a system:

1. *Checking requirements* These requirements identify checks on inputs to the system to ensure that incorrect or out-of-range inputs are detected before they are processed by the system.
2. *Recovery requirements* These requirements are geared to helping the system recover after a failure has occurred. Typically, these requirements are concerned with maintaining copies of the system and its data and specifying how to restore system services after a failure.
3. *Redundancy requirements* These specify redundant features of the system that ensure that a single component failure does not lead to a complete loss of service. I discuss this in more detail in the next chapter.

In addition, the reliability requirements may include process requirements for reliability. These are requirements to ensure that good practice, known to reduce the number of faults in a system, is used in the development process. Some examples of functional reliability and process requirements are shown in Figure 12.8.

There are no simple rules for deriving functional reliability requirements. In organizations that develop critical systems, there is usually organizational knowledge about possible reliability requirements and how these impact the actual reliability of a system. These organizations may specialize in specific types of system such as railway control systems, so the reliability requirements can be reused across a range of systems.

12.4 Security specification

The specification of security requirements for systems has something in common with safety requirements. It is impractical to specify them quantitatively, and security requirements are often 'shall not' requirements that define unacceptable system behavior rather than required system functionality. However, security is a more challenging problem than safety, for a number of reasons:

1. When considering safety, you can assume that the environment in which the system is installed is not hostile. No one is trying to cause a safety-related incident. When considering security, you have to assume that attacks on the system are deliberate and that the attacker may have knowledge of system weaknesses.
2. When system failures occur that pose a risk to safety, you look for the errors or omissions that have caused the failure. When deliberate attacks cause system failures, finding the root cause may be more difficult as the attacker may try to conceal the cause of the failure.
3. It is usually acceptable to shut down a system or to degrade system services to avoid a safety-related failure. However, attacks on a system may be so-called denial of service attacks, which are intended to shut down the system. Shutting down the system means that the attack has been successful.
4. Safety-related events are not generated by an intelligent adversary. An attacker can probe a system's defenses in a series of attacks, modifying the attacks as he or she learns more about the system and its responses.

These distinctions mean that security requirements usually have to be more extensive than safety requirements. Safety requirements lead to the generation of functional system requirements that provide protection against events and faults that could cause safety-related failures. They are mostly concerned with checking for problems and taking actions if these problems occur. By contrast, there are many types of security requirements that cover the different threats faced by a system. Firesmith (2003) has identified 10 types of security requirements that may be included in a system specification:

1. Identification requirements specify whether or not a system should identify its users before interacting with them.
2. Authentication requirements specify how users are identified.
3. Authorization requirements specify the privileges and access permissions of identified users.
4. Immunity requirements specify how a system should protect itself against viruses, worms, and similar threats.
5. Integrity requirements specify how data corruption can be avoided.



Security risk management

Safety is a legal issue and businesses cannot decide to opt out of producing safe systems. However, some aspects of security are business issues—a business can decide not to implement some security measures and to cover the losses that may result from this decision. Risk management is the process of deciding what assets must be protected and how much can be spent on protecting them.

http://www.SoftwareEngineering-9.com/Web/Security/RiskMan.html

6. Intrusion detection requirements specify what mechanisms should be used to detect attacks on the system.
7. Non-repudiation requirements specify that a party in a transaction cannot deny its involvement in that transaction.
8. Privacy requirements specify how data privacy is to be maintained.
9. Security auditing requirements specify how system use can be audited and checked.
10. System maintenance security requirements specify how an application can prevent authorized changes from accidentally defeating its security mechanisms.

Of course, you will not see all of these types of security requirements in every system. The particular requirements depend on the type of system, the situation of use, and the expected users.

The risk analysis and assessment process discussed in Section 12.1 may be used to identify system security requirements. As I discussed, there are three stages to this process:

1. *Preliminary risk analysis* At this stage, decisions on the detailed system requirements, the system design, or the implementation technology have not been made. The aim of this assessment process is to derive security requirements for the system as a whole.
2. *Life-cycle risk analysis* This risk assessment takes place during the system development life cycle after design choices have been made. The additional security requirements take account of the technologies used in building the system and system design and implementation decisions.
3. *Operational risk analysis* This risk assessment considers the risks posed by malicious attacks on the operational system by users, with or without insider knowledge of the system.

The risk assessment and analysis processes used in security requirements specification are variants of the generic risk-driven specification process discussed in

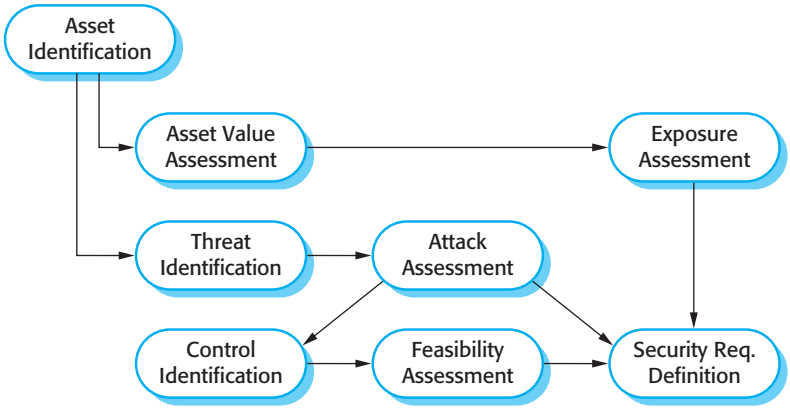


Figure 12.9 The preliminary risk assessment process for security requirements

Section 12.1. A risk-driven security requirements process is shown in Figure 12.9. This may appear to be different from the risk-driven process in Figure 12.1, but I indicate how each stage corresponds to stages in the generic process by including the generic process activity in brackets. The process stages are:

1. Asset identification, where the system assets that may require protection are identified. The system itself or particular system functions may be identified as assets as well as the data associated with the system (risk identification).
2. Asset value assessment, where you estimate the value of the identified assets (risk analysis).
3. Exposure assessment, where you assess the potential losses associated with each asset. This should take into account direct losses such as the theft of information, the costs of recovery, and the possible loss of reputation (risk analysis).
4. Threat identification, where you identify the threats to system assets (risk analysis).
5. Attack assessment, where you decompose each threat into attacks that might be made on the system and the possible ways in which these attacks may occur. You may use attack trees (Schneier, 1999) to analyze the possible attacks. These are similar to fault trees as you start with a threat at the root of the tree and identify possible causal attacks and how these might be made (risk decomposition).
6. Control identification, where you propose the controls that might be put in place to protect an asset. The controls are the technical mechanisms, such as encryption, that you can use to protect assets (risk reduction).
7. Feasibility assessment, where you assess the technical feasibility and the costs of the proposed controls. It is not worth having expensive controls to protect assets that don't have a high value (risk reduction).
8. Security requirements definition, where knowledge of the exposure, threats, and control assessments is used to derive system security requirements. These may

Asset	Value	Exposure
The information system	High. Required to support all clinical consultations. Potentially safety-critical.	High. Financial loss as clinics may have to be canceled. Costs of restoring system. Possible patient harm if treatment cannot be prescribed.
The patient database	High. Required to support all clinical consultations. Potentially safety-critical.	High. Financial loss as clinics may have to be canceled. Costs of restoring system. Possible patient harm if treatment cannot be prescribed.
An individual patient record	Normally low although may be high for specific high-profile patients.	Low direct losses but possible loss of reputation.

Figure 12.10 Asset analysis in a preliminary risk assessment report for the MHC-PMS

be requirements for the system infrastructure or the application system (risk reduction).

An important input to the risk assessment and management process is the organizational security policy. An organizational security policy applies to all systems and should set out what should and what should not be allowed. For example, one aspect of a military security policy may state 'Readers may only examine documents whose classification is the same as or below the reader's vetting level.' This means that if a reader has been vetted to a 'secret' level, they may access documents that are classed as 'secret,' 'confidential,' or 'open' but not documents classed as 'top secret.'

The security policy sets out conditions that should always be maintained by a security system and so helps identify threats that might arise. Threats are anything that could threaten business security. In practice, security policies are usually informal documents that define what is and what isn't allowed. However, Bishop (2005) discusses the possibility of expressing security policies in a formal language and generating automated checks to ensure that the policy is being followed.

To illustrate this process of security risk analysis, consider the hospital information system for mental health care, MHC-PMS. I don't have space to discuss a complete risk assessment here but rather draw on this system as a source of examples. I have shown these as a fragment of a report (Figures 12.10 and 12.11) that might be generated from the preliminary risk assessment process. This preliminary risk analysis report is used in defining the security requirements.

From the risk analysis for the hospital information system, you can derive security requirements. Some examples of these requirements are:

1. Patient information shall be downloaded, at the start of a clinic session, from the database to a secure area on the system client.

Threat	Probability	Control	Feasibility
Unauthorized user gains access as system manager and makes system unavailable	Low	Only allow system management from specific locations that are physically secure.	Low cost of implementation but care must be taken with key distribution and to ensure that keys are available in the event of an emergency.
Unauthorized user gains access as system user and accesses confidential information	High	Require all users to authenticate themselves using a biometric mechanism. Log all changes to patient information to track system usage.	Technically feasible but high-cost solution. Possible user resistance. Simple and transparent to implement and also supports recovery.

Figure 12.11 Threat and control analysis in a preliminary risk assessment report

2. All patient information on the system client shall be encrypted.
3. Patient information shall be uploaded to the database when a clinic session is over and deleted from the client computer.
4. A log of all changes made to the system database and the initiator of these changes shall be maintained on a separate computer from the database server.

The first two requirements are related—patient information is downloaded to a local machine so that consultations may continue if the patient database server is attacked or becomes unavailable. However, this information must be deleted so that later users of the client computer cannot access the information. The fourth requirement is a recovery and auditing requirement. It means that changes can be recovered by replaying the change log and that it is possible to discover who has made the changes. This accountability discourages misuse of the system by authorized staff.

12.5 Formal specification

For more than 30 years, many researchers have advocated the use of formal methods of software development. Formal methods are mathematically-based approaches to software development where you define a formal model of the software. You may then formally analyze this model and use it as a basis for a formal system specification. In principle, it is possible to start with a formal model for the software and prove that a developed program is consistent with that model, thus eliminating software failures resulting from programming errors.



Formal specification techniques

Formal system specifications may be expressed using two fundamental approaches, either as models of the system interfaces (algebraic specifications) or as models of the system state. You can download an extra web chapter on this topic, where I show examples of both of these approaches. The chapter includes a formal specification of part of the insulin pump system.

http://www.SoftwareEngineering-9.com/Web/ExtraChaps/FormalSpec.pdf

The starting point for all formal development processes is a formal system model, which serves as a system specification. To create this model, you translate the system's user requirements, which are expressed in natural language, diagrams, and tables, into a mathematical language which has formally defined semantics. The formal specification is an unambiguous description of what the system should do. Using manual or tool-supported methods, you can check that a program's behavior is consistent with the specification.

Formal specifications are not just essential for a verification of the design and implementation of software. They are the most precise way of specifying systems, and so reduce the scope for misunderstanding. Furthermore, constructing a formal specification forces a detailed analysis of the requirements and this is an effective way of discovering requirements problems. In a natural language specification, errors can be concealed by the imprecision of the language. This is not the case if the system is formally specified.

Formal specifications are usually developed as part of a plan-based software process, where the system is completely specified before development. The system requirements and design are defined in detail and are carefully analyzed and checked before implementation begins. If a formal specification of the software is developed, this usually comes after the system requirements have been specified but before the detailed system design. There is a tight feedback loop between the detailed requirements specification and the formal specification.

Figure 12.12 shows the stages of software specification and its interface with software design in a plan-based software process. As it is expensive to develop formal specifications, you may decide to limit the use of this approach to those components that are critical to the system's operation. You identify these in the architectural design of the system.

Over the past few years, automated support for analyzing a formal specification has been developed. Model checkers (Clarke et al., 2000) are software tools that take a state-based formal specification (a system model) as an input, along with the specification of some formally expressed desirable property, such as 'there are no unreachable states.' The model checking program exhaustively analyzes the specification and either reports that the system property is satisfied by the model or presents an example that shows it is not satisfied. Model checking is closely related to the notion of static analysis and I discuss these general approaches to system verification in Chapter 15.

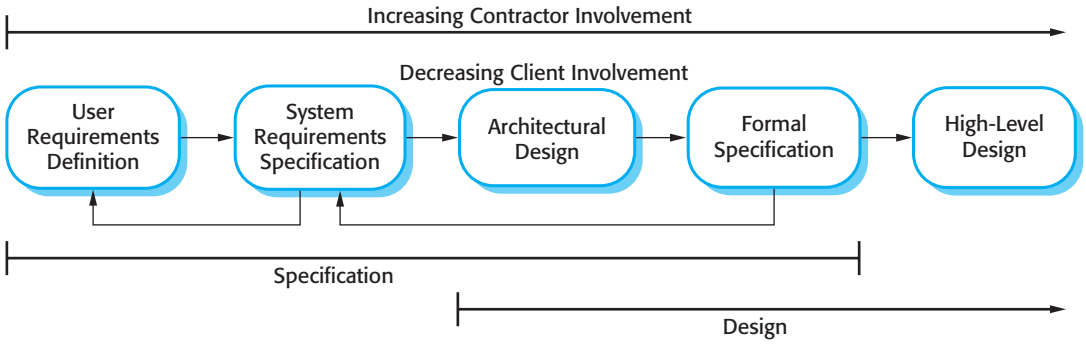


Figure 12.12 Formal specification in a plan-based software process

The advantages of developing a formal specification and using this in a formal development process are:

1. As you develop a formal specification in detail, you develop a deep and detailed understanding of the system requirements. Even if you do not use the specification in a formal development process, requirements error detection is a potent argument for developing a formal specification (Hall, 1990). Requirements problems that are discovered early are usually much cheaper to correct than if they are found at later stages in the development process.
2. As the specification is expressed in a language with formally defined semantics, you can analyze it automatically to discover inconsistencies and incompleteness.
3. If you use a method such as the B method, you can transform the formal specification into a program through a sequence of correctness-preserving transformations. The resulting program is therefore guaranteed to meet its specification.
4. Program testing costs may be reduced because you have verified the program against its specification.

In spite of these advantages, formal methods have had limited impact on practical software development, even for critical systems. Consequently, there is very little experience in the community of developing and using formal system specifications. The arguments that are put forward against developing a formal system specification are:

1. Problem owners and domain experts cannot understand a formal specification so they cannot check that it accurately represents their requirements. Software engineers, who understand the formal specification, may not understand the application domain so they too cannot be sure that the formal specification is an accurate reflection of the system requirements.
2. It is fairly easy to quantify the costs of creating a formal specification, but more difficult to estimate the possible cost savings that will result from its use. As a result, managers are unwilling to take the risk of adopting this approach.



Formal specification costs

Developing a formal specification is an expensive process as quite a lot of time is needed to translate the requirements into a formal language and check the specification. Experience has shown that savings can be made in system testing and verification and it seems that specifying a system formally does not significantly increase the overall development costs. However, the balance of costs changes, with more costs incurred early in the development process.

http://www.SoftwareEngineering-9.com/Web/FormalSpecCosts/

3. Most software engineers have not been trained to use formal specification languages. Hence, they are reluctant to propose their use in development processes.
4. It is difficult to scale current approaches to formal specification up to very large systems. When formal specification is used, it is mostly for specifying critical kernel software rather than complete systems.
5. Formal specification is not compatible with agile methods of development.

Nevertheless, at the time of writing, formal methods have been used in the development of a number of safety- and security-critical applications. They may also be used cost effectively in the development and validation of critical parts of a larger, more complex software system (Badeau and Amelot, 2005; Hall, 1996; Hall and Chapman, 2002; Miller et al., 2005; Wordworth, 1996). They are the basis of tools used in static verification such as the driver verification system used by Microsoft (Ball et al., 2004; Ball et al., 2006) and the SPARK/Ada language (Barnes, 2003) for critical systems engineering.

KEY POINTS

- Risk analysis is an important activity in the specification of security and dependability requirements. It involves identifying risks that can result in accidents or incidents. System requirements are then generated to ensure that these risks do not occur and, if they do, that they do not lead to an incident or accident.
- A hazard-driven approach may be used to understand the safety requirements for a system. You identify potential hazards and decompose these (using methods such as fault tree analysis) to discover their root causes. You then specify requirements to avoid or recover from these problems.
- Reliability requirements can be defined quantitatively in the system requirements specification. Reliability metrics include probability of failure on demand (POFOD), rate of occurrence of failure (ROCOF), and availability (AVAIL).

- It is important not to overspecify the required system reliability as this leads to unnecessary additional costs in the development and validation processes.
- Security requirements are more difficult to identify than safety requirements because a system attacker can use knowledge of system vulnerabilities to plan a system attack, and can learn about vulnerabilities from unsuccessful attacks.
- To specify security requirements, you should identify the assets that are to be protected and define how security techniques and technology should be used to protect these assets.
- Formal methods of software development rely on a system specification that is expressed as a mathematical model. Developing a formal specification has the key benefit of stimulating a detailed examination and analysis of the system requirements.

FURTHER READING

Safeware: System Safety and Computers. This is a thorough discussion of all aspects of safety-critical systems. It is particularly strong in its description of hazard analysis and the derivation of requirements from this. (N. Leveson, Addison-Wesley, 1995.)

'Security Use Cases.' A good article, available on the Web, that focuses on how use cases can be used in security specification. The author also has a number of good articles on security specification that are referenced in this article. (D. G. Firesmith, *Journal of Object Technology*, **2** (3), May–June 2003.) http://www.jot.fm/issues/issue_2003_05/column6/.

'Ten Commandments of Formal Methods . . .Ten Years Later.' This is a set of guidelines for the use of formal methods that was first proposed in 1996 and which are revisited in this paper. It is a good summary of the practical issues around the use of formal methods. (J. P. Bowen and M. G. Hinchey, *IEEE Computer*, **39** (1), January 2006.) http://dx.doi.org/10.1109/MC.2006.35.

'Security Requirements for the Rest of Us: A Survey.' A good starting point for reading about security requirements specification. The authors focus on lightweight rather than formal approaches. (I. A. Tøndel, M. G. Jaatun, and P. H. Meland, *IEEE Software*, **25** (1), January/February 2008.) http://dx.doi.org/10.1109/MS.2008.19.

EXERCISES

12.1. Explain why the boundaries in the risk triangle shown in Figure 12.12 are liable to change with time and changing social attitudes.

12.2. Explain why the risk-based approach is interpreted in different ways when specifying safety and security.

12.3. In the insulin pump system, the user has to change the needle and insulin supply at regular intervals and may also change the maximum single dose and the maximum daily dose that may be administered. Suggest three user errors that might occur and propose safety requirements that would avoid these errors resulting in an accident.

12.4. A safety-critical software system for treating cancer patients has two principal components:

- A radiation therapy machine that delivers controlled doses of radiation to tumor sites. This machine is controlled by an embedded software system.
- A treatment database that includes details of the treatment given to each patient. Treatment requirements are entered in this database and are automatically downloaded to the radiation therapy machine.

Identify three hazards that may arise in this system. For each hazard, suggest a defensive requirement that will reduce the probability that these hazards will result in an accident. Explain why your suggested defense is likely to reduce the risk associated with the hazard.

12.5. Suggest appropriate reliability metrics for the classes of software systems below. Give reasons for your choice of metric. Predict the usage of these systems and suggest appropriate values for the reliability metrics.

- a system that monitors patients in a hospital intensive care unit
- a word processor
- an automated vending machine control system
- a system to control braking in a car
- a system to control a refrigeration unit
- a management report generator

12.6. A train protection system automatically applies the brakes of a train if the speed limit for a segment of track is exceeded, or if the train enters a track segment that is currently signaled with a red light (i.e., the segment should not be entered). Giving reasons for your answer, choose a reliability metric that might be used to specify the required reliability for such a system.

12.7. There are two essential safety requirements for the train protection system:

- The train shall not enter a segment of track that is signaled with a red light.
- The train shall not exceed the specified speed limit for a section of track.

Assuming that the signal status and the speed limit for the track segment are transmitted to onboard software on the train before it enters the track segment, propose five possible functional system requirements for the onboard software that may be generated from the system safety requirements.

12.8. Explain why there is a need for both preliminary security risk assessment and life-cycle security risk assessment during the development of a system.

12.9. Extend the table in Figure 12.11 to identify two further threats to the MHC-PMS, along with associated controls. Use these as a basis for generating further software security requirements that implement the proposed controls.

12.10. Should software engineers working on the specification and development of safety-related systems be professionally certified in some way? Explain your reasoning.

REFERENCES

Badeau, F. and Amelot, A. (2005). 'Using B as a High Level Programming Language in an Industrial Project: Roissy VAL'. Proc. ZB 2005: Formal Specification and Development in Z and B, Guildford, UK: Springer.

Ball, T., Bounimova, E., Cook, B., Levin, V., Lichtenberg, J., McGarvey, C., Ondrusek, B., Rajamani, S. K. and Ustuner, A. (2006). 'Thorough Static Analysis of Device Drivers'. Proc. EuroSys 2006, Leuven, Belgium.

Ball, T., Cook, B., Levin, V. and Rajamani, S. K. (2004). 'SLAM and Static Driver Verifier: Technology Transfer of Formal Methods Inside Microsoft'. Proc. Integrated Formal Methods 2004, Canterbury, UK: Springer.

Barnes, J. P. (2003). *High-integrity Software: The SPARK Approach to Safety and Security*. Harlow, UK: Addison-Wesley.

Bishop, M. (2005). *Introduction to Computer Security*. Boston: Addison-Wesley.

Brazendale, J. and Bell, R. (1994). 'Safety-related control and protection systems: standards update'. *IEE Computing and Control Engineering J.*, **5** (1), 6–12.

Clarke, E. M., Grumberg, O. and Peled, D. A. (2000). *Model Checking*. Cambridge, Mass.: MIT Press.

Firesmith, D. G. (2003). 'Engineering Security Requirements'. *Journal of Object Technology*, **2** (1), 53–68.

Hall, A. (1990). 'Seven Myths of Formal Methods'. *IEEE Software*, **7** (5), 11–20.

Hall, A. (1996). 'Using Formal methods to Develop an ATC Information System'. *IEEE Software*, **13** (2), 66–76.

Hall, A. and Chapman, R. (2002). 'Correctness by Construction: Developing a Commercially Secure System'. *IEEE Software*, **19** (1), 18–25.

Jahanian, F. and Mok, A. K. (1986). 'Safety analysis of timing properties in real-time systems'. *IEEE Trans.on Software Engineering.*, **SE-12** (9), 890–904.

Leveson, N. and Stolzy, J. (1987). 'Safety analysis using Petri nets'. *IEEE Transactions on Software Engineering*, **13** (3), 386–397.

Leveson, N. G. (1995). *Safeware: System Safety and Computers*. Reading, Mass.: Addison-Wesley.

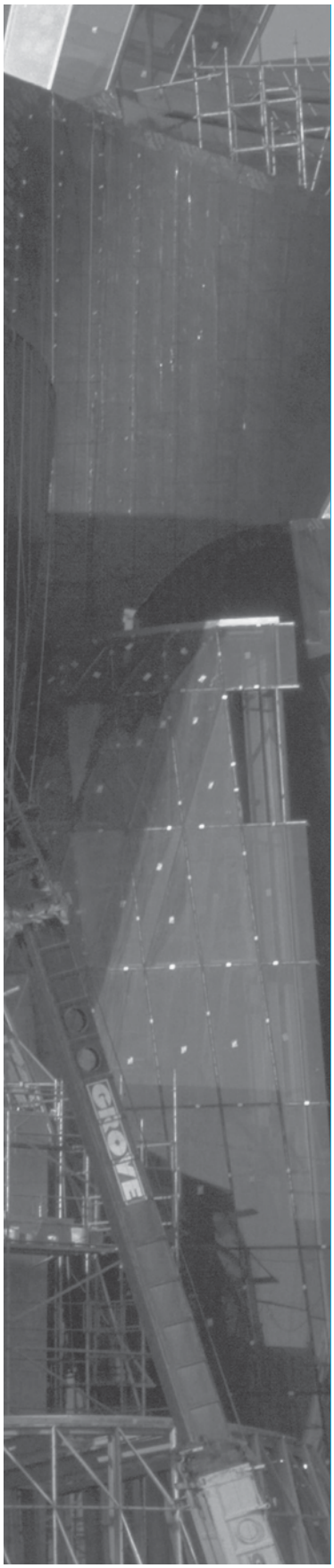
Miller, S. P., Anderson, E. A., Wagner, L. G., Whalen, M. W. and Heimdahl, M. P. E. (2005). 'Formal Verification of Flight Control Software'. *Proc. AIAA Guidance*, Navigation and Control Conference, San Francisco.

Peterson, J. L. (1981). *Petri Net Theory and the Modeling of Systems*. New York: McGraw-Hill.

Schneier, B. (1999). 'Attack Trees'. *Dr Dobbs Journal*, **24** (12), 1–9.

Storey, N. (1996). *Safety-Critical Computer Systems*. Harlow, UK: Addison-Wesley.

Wordsworth, J. (1996). *Software Engineering with B*. Wokingham: Addison-Wesley.



13

Dependability engineering

Objectives

The objective of this chapter is to discuss processes and techniques for developing highly dependable systems. When you have read this chapter you will:

- understand how system dependability can be achieved by using redundant and diverse components;
- know how dependable software processes contribute to the development of dependable software;
- understand how different architectural styles may be used to implement software redundancy and diversity;
- be aware of good programming practice that should be used in dependable systems engineering.

Contents

13.1 Redundancy and diversity
13.2 Dependable processes
13.3 Dependable system architectures
13.4 Dependable programming

The use of software engineering techniques, better programming languages, and better quality management has led to significant improvements in dependability for most software. Nevertheless, system failures may still occur that affect the system's availability or lead to incorrect results being produced. In some cases, these failures simply cause minor inconvenience. System vendors may simply decide to live with these failures, without correcting the errors in their systems. However, in some systems, failure can lead to loss of life or significant economic or reputational losses. These are known as 'critical systems', for which a high level of dependability is essential.

Examples of critical systems include process control systems, protection systems that shut down other systems in the event of failure, medical systems, telecommunications switches, and flight control systems. Special development tools and techniques may be used to enhance the dependability of the software in a critical system. These tools and techniques usually increase the costs of system development but they reduce the risk of system failure and the losses that may result from such a failure.

Dependability engineering is concerned with the techniques that are used to enhance the dependability of both critical and non-critical systems. These techniques support three complementary approaches that are used in developing dependable software:

1. *Fault avoidance* The software design and implementation process should use approaches to software development that help avoid design and programming errors and so minimize the number of faults that are likely to arise when the system is executing. Fewer faults mean less chance of run-time failures.
2. *Fault detection and correction* The verification and validation processes are designed to discover and remove faults in a program, before it is deployed for operational use. Critical systems require very extensive verification and validation to discover as many faults as possible before deployment and to convince the system stakeholders that the system is dependable. I cover this topic in Chapter 15.
3. *Fault tolerance* The system is designed so that faults or unexpected system behavior during execution are detected at run-time and are managed in such a way that system failure does not occur. Simple approaches to fault tolerance based on built-in run-time checking may be included in all systems. However, more specialized fault-tolerance techniques (such as the use of fault-tolerant system architectures) are generally only used when a very high level of system availability and reliability is required.

Unfortunately, applying fault-avoidance, fault-detection, and fault-tolerance techniques leads to a situation of diminishing returns. The cost of finding and removing the remaining faults in a software system rises exponentially as program faults are discovered and removed (Figure 13.1). As the software becomes more reliable, you need to spend more and more time and effort to find fewer and fewer faults. At some stage, even for critical systems, the costs of this additional effort become unjustifiable.

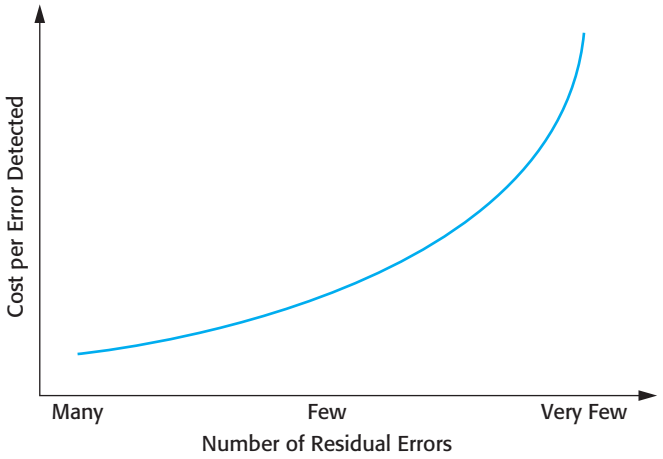


Figure 13.1 The increasing costs of residual fault removal

As a result, software development companies accept that their software will always contain some residual faults. The level of faults depends on the type of system. Shrink-wrapped products have a relatively high level of faults, whereas critical systems usually have a much lower fault density.

The rationale for accepting faults is that, if and when the system fails, it is cheaper to pay for the consequences of failure than it would be to discover and remove the faults before system delivery. However, as discussed in Chapter 11, the decision to release faulty software is not simply an economic decision. The social and political acceptability of system failure must also be taken into account.

Many critical systems, such as aircraft systems, medical systems, and accounting systems, are used in regulated domains such as air transport, medicine, and finance. National governments define regulations that apply in these domains and appoint a regulatory body to ensure that companies follow these regulations. In practice, this means that the regulator often has to be convinced that critical software systems can be trusted and this requires clear evidence that shows that these systems are dependable.

Therefore, the development process for critical systems is not just concerned with producing a dependable system; it must also produce the evidence that can convince a regulator that the system is dependable. Producing such evidence consumes a high proportion of the development costs for critical systems and so is an important contributory factor to the high costs of critical systems. I discuss the issues of producing safety and dependability cases in Chapter 15.

13.1 Redundancy and diversity

Redundancy and diversity are fundamental strategies for enhancing the dependability of any type of system. Redundancy means that spare capacity is included in a system that can be used if part of that system fails. Diversity means that redundant



The Ariane 5 explosion

In 1996, the European Space Agency's Ariane 5 rocket exploded 37 seconds after liftoff on its maiden flight. The fault was caused by a software systems failure. There was a backup system but this was not diverse and so the software in the backup computer failed in exactly the same way. The rocket and its satellite payload were destroyed.

http://www.SoftwareEngineering-9.com/Web/DependabilityEng/Ariane/

components of the system are of different types, thus increasing the chances that they will not fail in exactly the same way.

We use redundancy and diversity to enhance dependability in our everyday lives. As an example of redundancy, most people keep spare light bulbs in their homes so that they can quickly recover from the failure of a light bulb that is in use. Commonly, to secure our homes we use more than one lock (redundancy) and, usually, the locks used are of different types (diversity). This means that if an intruder finds a way to defeat one of the locks, they have to find a different way of defeating the other lock before they can gain entry. As a matter of routine, we should all back up our computers and so maintain redundant copies of our data. To avoid problems with disk failure, backups should be kept on a separate, diverse, external device.

Software systems that are designed for dependability may include redundant components that provide the same functionality as other system components. These are switched into the system if the primary component fails. If these redundant components are diverse (i.e., not the same as other components), a common fault in replicated components will not result in a system failure. Redundancy may also be provided by including additional checking code, which is not strictly necessary for the system to function. This code can detect some kinds of faults before they cause failures. It can invoke recovery mechanisms to ensure that the system continues to operate.

In systems for which availability is a critical requirement, redundant servers are normally used. These automatically come into operation if a designated server fails. Sometimes, to ensure that attacks on the system cannot exploit a common vulnerability, these servers may be of different types and may run different operating systems. Using different operating systems is one example of software diversity and redundancy, where comparable functionality is provided in different ways. I discuss software diversity in more detail in Section 13.3.4.

Diversity and redundancy may also be also used to achieve dependable processes by ensuring that process activities, such as software validation, do not rely on a single process or method. This improves software dependability because it reduces the chances of process failure, where human errors made during the software development process lead to software errors. For example, validation activities may include program testing, manual program inspections, and static analysis as fault-finding techniques. These are complementary techniques in that any one technique might find faults that are missed by the other methods. Furthermore, different team members may be responsible for the same process activity (e.g., a program inspection).



Dependable operational processes

This chapter discusses dependable development processes but an equally important contributor to system dependability is a system's operational processes. In designing these operational processes, you have to take into account human factors and always bear in mind that people are liable to make mistakes when using a system. A dependable process should be designed to avoid human errors and, when mistakes are made, the software should detect the mistakes and allow them to be corrected.

http://www.SoftwareEngineering-9.com/Web/DependabilityEng/HumanFactors/

People tackle tasks in different ways depending on their personality, experience, and education, so this kind of redundancy provides a diverse perspective on the system.

As I discuss in Section 13.3.4, achieving software diversity is not straightforward. Diversity and redundancy make systems more complex and usually harder to understand. Not only is there more code to write and check, additional functionality must also be added to the system to detect component failure and to switch control to alternative components. This additional complexity means that it is more likely that programmers will make errors and less likely that people checking the system will find these errors.

As a consequence, some people think that it is best to avoid software redundancy and diversity. Their view is that the best approach is to design the software to be as simple as possible, with extremely rigorous software verification and validation procedures (Parnas et al., 1990). More can be spent on verification and validation because of the savings that result from not having to develop redundant software components.

Both approaches are used in commercial, safety-critical systems. For example, the Airbus 340 flight control hardware and software is both diverse and redundant (Storey, 1996). The flight control software on the Boeing 777 is based on a redundant hardware but each computer runs the same software, which has been extensively validated. The Boeing 777 flight control system designers have focused on simplicity rather than redundancy. Both of these aircraft are very reliable, so both the diverse and the simple approach to dependability can clearly be successful.

13.2 Dependable processes

Dependable software processes are software processes that are designed to produce dependable software. A company using a dependable process can be sure that the process has been properly enacted and documented and that appropriate development techniques have been used for critical systems development. The rationale for investing in dependable processes is that a good software process is likely to lead to delivered software that contains fewer errors and is therefore less likely to fail in execution. Figure 13.2 shows some of the attributes of dependable software processes.

The evidence that a dependable process has been used is often important in convincing a regulator that the most effective software engineering practice has been applied in developing the software. System developers will normally present a model of the process to a regulator, along with evidence that the process has been

Process Characteristic	Description
Documentable	The process should have a defined process model that sets out the activities in the process and the documentation that is to be produced during these activities.
Standardized	A comprehensive set of software development standards covering software production and documentation should be available.
Auditable	The process should be understandable by people apart from process participants, who can check that process standards are being followed and make suggestions for process improvement.
Diverse	The process should include redundant and diverse verification and validation activities.
Robust	The process should be able to recover from failures of individual process activities.

Figure 13.2 Attributes of dependable processes

followed. The regulator also has to be convinced that the process is used consistently by all of the process participants and that it can be used in different development projects. This means that the process must be explicitly defined and repeatable:

1. An explicitly defined process is one that has a defined process model that is used to drive the software production process. There must be data collected during the process that demonstrates that all of the necessary steps in the process model have been enacted.
2. A repeatable process is one that does not rely on individual interpretation and judgment. Rather, the process can be repeated across projects and with different team members, irrespective of who is involved in the development. This is particularly important for critical systems, which often have a long development cycle during which there are often significant changes in the development team.

Dependable processes make use of redundancy and diversity to achieve reliability. They often include different activities that have the same aim. For example, program inspections and testing aim to discover errors in a program. The approaches are complementary so that together they are likely to discover a higher proportion of errors than would be found using one technique on its own.

The activities that are used in dependable processes obviously depend on the type of software that is being developed. In general, however, these activities should be geared to avoiding the introduction of errors into a system, detecting and removing errors, and maintaining information about the process itself. Examples of activities that might be included in a dependable process include:

1. Requirements reviews to check that the requirements are, as far as possible, complete and consistent.



The safety life cycle

The International Electrotechnical Commission has devised a process standard (IEC 61508) for protection systems engineering. This is based around the notion of a safety life cycle, which makes a clear distinction between safety engineering and system engineering. The first stages of the IEC 61508 safety life cycle define the scope of the system, assess the potential system hazards, and estimate the risks they pose. This is followed by the specification of the safety requirements and the allocation of these safety requirements to different subsystems. The idea is to limit the extent of safety-critical functionality to allow specific techniques for critical systems engineering to be applied to the development of the safety-critical system.

http://www.SoftwareEngineering-9.com/Web/SafetyLifeCycle/

2. Requirements management to ensure that changes to the requirements are controlled and that the impact of proposed requirements changes is understood by all developers affected by the change.
3. Formal specification, where a mathematical model of the software is created and analyzed. I discussed the benefits of formal specification in Chapter 12. Perhaps its most important benefit is that it forces a very detailed analysis of the system requirements. This analysis itself is likely to discover requirements problems that may have been missed in requirements reviews.
4. System modeling, where the software design is explicitly documented as a set of graphical models, and the links between the requirements and these models are explicitly documented.
5. Design and program inspections, where the different descriptions of the system are inspected and checked by different people. Inspections are often driven by checklists of common design and programming errors.
6. Static analysis, where automated checks are carried out on the source code of the program. These look for anomalies that could indicate programming errors or omissions. I discuss static analysis in Chapter 15.
7. Test planning and management, where a comprehensive set of system tests is designed. The testing process has to be carefully managed to demonstrate that these tests provide coverage of the system requirements and have been correctly applied in the testing process.

As well as process activities that focus on system development and testing, there must also be well-defined quality management and change management processes. Although the specific activities in a dependable process may vary from one company to another, the need for effective quality and change management is universal.

Quality management processes (discussed in Chapter 24) establish a set of process and product standards. They also include activities that capture process information to demonstrate that these standards have been followed. For example, there may be a standard defined for carrying out program inspections. The inspection team leader is responsible for documenting the process to show that the inspection standard has been followed.

Change management, discussed in Chapter 25, is concerned with managing changes to a system, ensuring that accepted changes are actually implemented and confirming that planned releases of the software include the planned changes. One common problem with software is that the wrong components are included in a system build. This can lead to a situation where an executing system includes components that have not been checked during the development process. Configuration management procedures must be defined as part of the change management process to ensure that this does not happen.

There is a widely held view that agile approaches, as discussed in Chapter 3, are not really suitable for dependable processes (Boehm, 2002). Agile approaches focus on developing the software rather than on documenting what has been done. They often have a fairly informal approach to change and quality management. Plan-based approaches to dependable systems development, which create documentation that regulators and other external system stakeholders can understand, are generally preferred. Nevertheless, the benefits of agile approaches are equally applicable to critical systems. There have been reports of successes in applying agile methods in this area (Lindvall, et al., 2004) and it is likely that variants of agile methods that are suitable for critical systems engineering will be developed.

13.3 Dependable system architectures

As I have discussed, dependable systems development should be based around a dependable process. However, although you probably need a dependable process to create dependable systems, this is not enough in itself to ensure dependability. You also need to design a system architecture for dependability, especially when fault tolerance is required. This means that the architecture has to be designed to include redundant components and mechanisms that allow control to be switched from one component to another.

Examples of systems that may need fault-tolerant architectures are systems in aircraft that must be in operation throughout the duration of the flight, telecommunication systems, and critical command and control systems. Pullum (2001) describes different types of fault-tolerant architecture that have been proposed and Torres-Pomales surveys software fault-tolerance techniques (2000).

The simplest realization of a dependable architecture is in replicated servers, where two or more servers carry out the same task. Requests for processing are channeled through a server management component that routes each request to a particular server. This component also keeps track of server responses. In the event of server failure, which is usually detected by a lack of response, the faulty server is switched out of the system. Unprocessed requests are resubmitted to other servers for processing.

This replicated server approach is widely used for transaction processing systems where it is easy to maintain copies of transactions to be processed. Transaction processing systems are designed so that data is only updated once a transaction has finished correctly so delays in processing do not affect the integrity of the system.

It can be an efficient way of using hardware if the backup server is one that is normally used for low-priority tasks. If a problem occurs with a primary server, its processing is transferred to the backup server, which gives that work the highest priority.

Replicated servers provide redundancy but not usually diversity. The hardware is usually identical and they run the same version of the software. Therefore, they can cope with hardware failures and software failures that are localized to a single machine. They cannot cope with software design problems that cause all versions of the software to fail at the same time. To handle software design failures, a system has to include diverse software and hardware, as I have discussed in Section 13.1.

Software diversity and redundancy can be implemented in a number of different architectural styles. I describe some of these in the remainder of this section.

13.3.1 Protection systems

A protection system is a specialized system that is associated with some other system. This is usually a control system for some process, such as a chemical manufacturing process or an equipment control system, such as the system on a driverless train. An example of a protection system might be a system on a train that detects if the train has gone through a red signal. If so, and there is no indication that the train control system is decelerating the train, then the protection system automatically applies the train brakes to bring it to a halt. Protection systems independently monitor their environment and, if the sensors indicate a problem that the controlled system is not dealing with, then the protection system is activated to shut down the process or equipment.

Figure 13.3 illustrates the relationship between a protection system and a controlled system. The protection system monitors both the controlled equipment and the environment. If a problem is detected, it issues commands to the actuators to shut down the system or invoke other protection mechanisms such as opening a pressure-release valve. Notice that there are two sets of sensors. One set is used for normal system monitoring and the other specifically for the protection system. In the event of sensor failure, there are backups that will allow the protection system to continue in operation. There may also be redundant actuators in the system.

A protection system only includes the critical functionality that is required to move the system from a potentially unsafe state to a safe state (system shutdown). It is an instance of a more general fault-tolerant architecture in which a principal system is supported by a smaller and simpler backup system that only includes essential functionality. For example, the U.S. space shuttle control software has a backup system that includes ‘get you home’ functionality; that is, the backup system can land the vehicle if the principal control system fails.

The advantage of this kind of architecture is that protection system software can be much simpler than the software that is controlling the protected process. The only function of the protection system is to monitor operation and to ensure that the system is brought to a safe state in the event of an emergency. Therefore, it is possible to invest more effort in fault avoidance and fault detection. You can check that the software

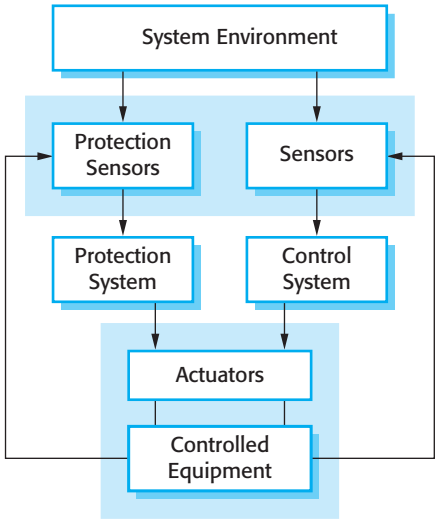


Figure 13.3 Protection system architecture

specification is correct and consistent and that the software is correct with respect to its specification. The aim is to ensure that the reliability of the protection system is such that it has a very low probability of failure on demand (say, 0.001). Given that demands on the protection system should be rare, a probability of failure on demand of 1/1,000 means that protection system failures should be very rare indeed.

13.3.2 Self-monitoring architectures

A self-monitoring architecture is a system architecture in which the system is designed to monitor its own operation and to take some action if a problem is detected. This is achieved by carrying out computations on separate channels and comparing the outputs of these computations. If the outputs are identical and are available at the same time, then it is judged that the system is operating correctly. If the outputs are different, then a failure is assumed. When this occurs, the system will normally raise a failure exception on the status output line, which will lead to control being transferred to another system. This is illustrated in Figure 13.4.

To be effective in detecting both hardware and software faults, self-monitoring systems have to be designed so that:

1. The hardware used in each channel is diverse. In practice, this might mean that each channel uses a different processor type to carry out the required computations, or the chipset making up the system may be sourced from different manufacturers. This reduces the probability of common processor design faults affecting the computation.
2. The software used in each channel is diverse. Otherwise, the same software error could arise at the same time on each channel. I discuss the difficulties of achieving truly diverse software in Section 13.3.4.

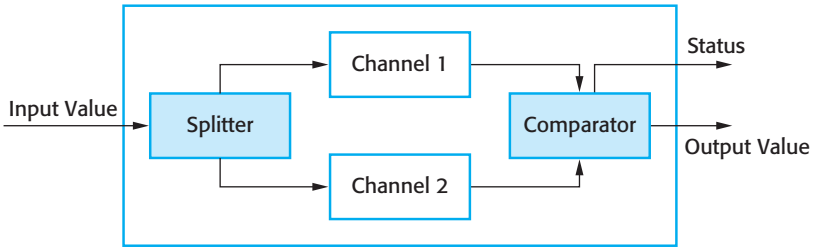


Figure 13.4 Self-monitoring architecture

On its own, this architecture may be used in situations where it is important for computations to be correct, but where availability is not essential. If the answers from each channel differ, the system simply shuts down. For many medical treatment and diagnostic systems, reliability is more important than availability as an incorrect system response could lead to the patient receiving incorrect treatment. However, if the system simply shuts down in the event of an error, this is an inconvenience but the patient will not usually be harmed by the system.

In situations where high availability is required, you have to use several self-checking systems in parallel. You need a switching unit that detects faults and selects a result from one of the systems, where both channels are producing a consistent response. Such an approach is used in the flight control system for the Airbus 340 series of aircraft, in which five self-checking computers are used. Figure 13.5 is a simplified diagram illustrating this organization.

In the Airbus flight control system, each of the flight control computers carry out the computations in parallel, using the same inputs. The outputs are connected to hardware filters that detect if the status indicates a fault and, if so, that the output from that computer is switched off. The output is then taken from an alternative system. Therefore, it is possible for four computers to fail and for the aircraft operation to continue. In more than 15 years of operation, there have been no reports of situations where control of the aircraft has been lost due to total flight control system failure.

The designers of the Airbus system have tried to achieve diversity in a number of different ways:

1. The primary flight control computers use a different processor from the secondary flight control systems.
2. The chipset that is used in each channel in the primary and secondary systems is supplied by a different manufacturer.
3. The software in the secondary flight control systems provides critical functionality only—it is less complex than the primary software.
4. The software for each channel in both the primary and the secondary systems is developed using different programming languages and by different teams.
5. Different programming languages are used in the secondary and primary systems.

As I discuss in the following section, these do not guarantee diversity but they reduce the probability of common failures in different channels.

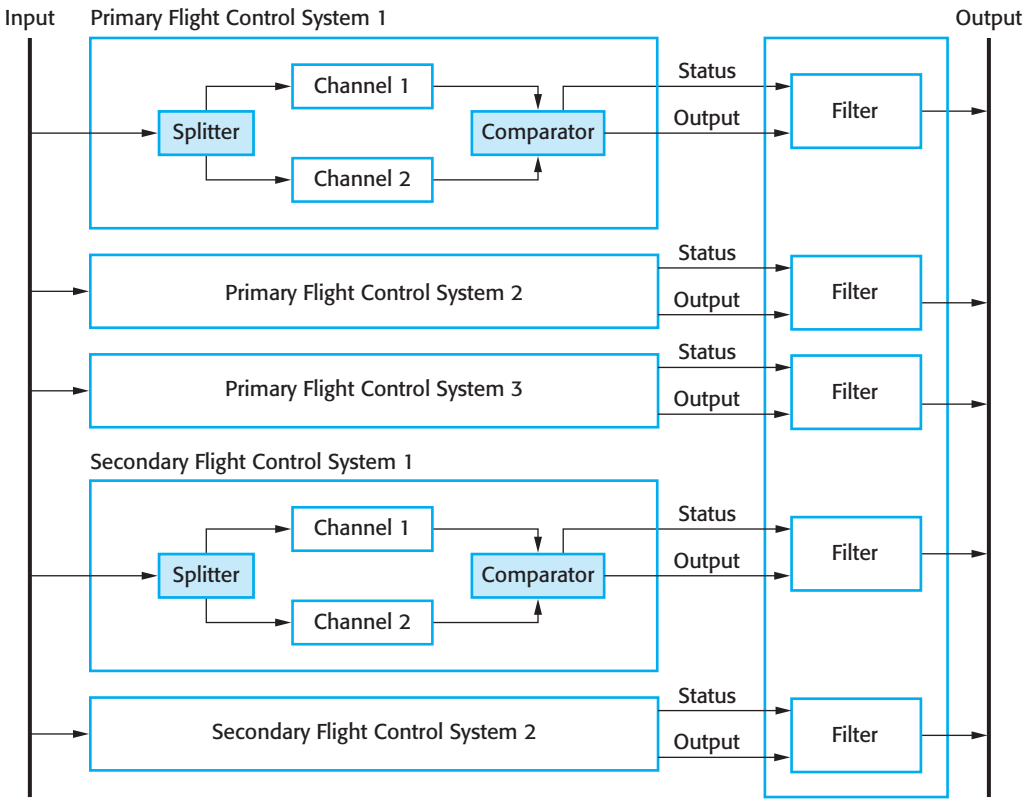


Figure 13.5 Airbus flight control system architecture

13.3.3 N-version programming

Self-monitoring architectures are examples of systems in which multiversion programming is used to provide software redundancy and diversity. This notion of multiversion programming has been derived from hardware systems where the notion of triple modular redundancy (TMR) has been used for many years to build systems that are tolerant of hardware failures (Figure 13.6).

In a TMR system, the hardware unit is replicated three (or sometimes more) times. The output from each unit is passed to an output comparator that is usually implemented as a voting system. This system compares all of its inputs and, if two or more are the same, then that value is output. If one of the units fails and does not produce the same output as the other units, its output is ignored. A fault manager may try to repair the faulty unit automatically but if this is impossible, the system is automatically reconfigured to take the unit out of service. The system then continues to function with two working units.

This approach to fault tolerance relies on most hardware failures being the result of component failure rather than design faults. The components are therefore likely to fail independently. It assumes that, when fully operational, all hardware units perform to specification. There is therefore a low probability of simultaneous component failure in all hardware units.

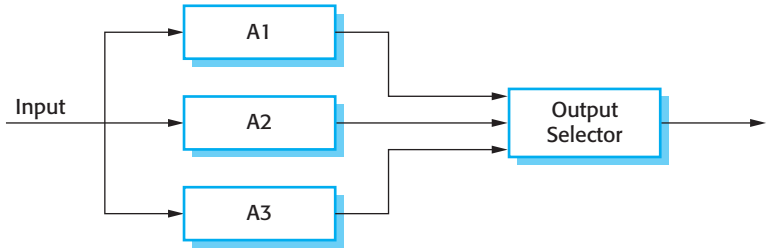


Figure 13.6 Triple modular redundancy

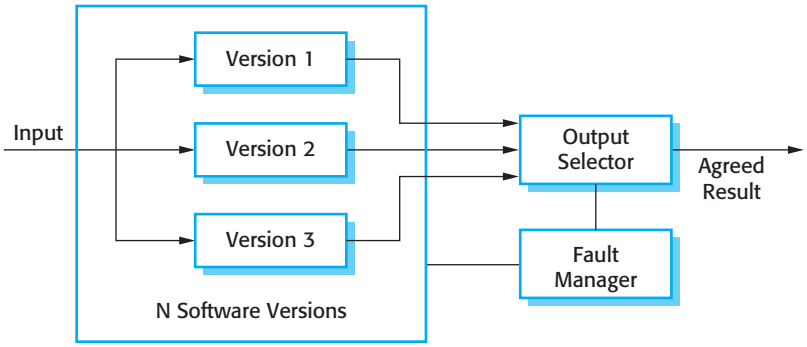


Figure 13.7 N-version programming

Of course, the components could all have a common design fault and thus all produce the same (wrong) answer. Using hardware units that have a common specification but which are designed and built by different manufacturers reduces the chances of such a common mode failure. It is assumed that the probability of different teams making the same design or manufacturing error is small.

A similar approach can be used for fault-tolerant software where N diverse versions of a software system execute in parallel (Avizienis, 1985; Avizienis,1995). This approach to software fault tolerance, illustrated in Figure 13.7, has been used in railway signaling systems, aircraft systems, and reactor protection systems.

Using a common specification, the same software system is implemented by a number of teams. These versions are executed on separate computers. Their outputs are compared using a voting system, and inconsistent outputs or outputs that are not produced in time are rejected. At least three versions of the system should be available so that two versions should be consistent in the event of a single failure.

N-version programming may be less expensive that self-checking architectures in systems for which a high level of availability is required. However, it still requires several different teams to develop different versions of the software. This leads to very high software development costs. As a result, this approach is only used in systems where it is impractical to provide a protection system that can guard against safety-critical failures.

13.3.4 Software diversity

All of the above fault-tolerant architectures rely on software diversity to achieve fault tolerance. This is based on the assumption that diverse implementations of the same specification (or a part of the specification, for protection systems) are independent. They should not include common errors and so will not fail in the same way, at the

same time. This requires the software to be written by different teams who should not communicate during the development process, therefore reducing the chances of common misunderstandings or misinterpretations of the specification.

The company that is procuring the system may include explicit diversity policies that are intended to maximize the differences between the system versions. For example:

1. By including requirements that different design methods should be used. For example, one team may be required to produce an object-oriented design and another team may produce a function-oriented design.
2. By stipulating that the implementations are to be written in different programming languages. For example, in a three-version system, Ada, C++, and Java could be used to write the software versions.
3. By requiring the use of different tools and development environments for the system.
4. By explicitly requiring different algorithms to be used in some parts of the implementation. However, this limits the freedom of the design team and may be difficult to reconcile with system performance requirements.

Each development team should work with a detailed system specification (sometimes called the *V-spec*) that has been derived from the system requirements specification (Avizienis, 1995). This should be sufficiently detailed to ensure that there are no ambiguities in the specification. As well as specifying the functionality of the system, the detailed specification should define where system outputs for comparison should be generated.

Ideally, the diverse versions of the system should have no dependencies and so should fail in completely different ways. If this is the case, then the overall reliability of a diverse system is obtained by multiplying the reliabilities of each channel. So, if each channel has a probability of failure on demand of 0.001, then the overall POFOD of a three-channel system (with all channels independent) is a million times greater than the reliability of a single-channel system.

In practice, however, achieving complete channel independence is impossible. It has been shown experimentally that independent design teams often make the same mistakes or misunderstand the same parts of the specification (Brilliant, et., 1990; Knight and Leveson, 1986; Leveson, 1995). There are several reasons for this:

1. Members of different teams are often from the same cultural background and may have been educated using the same approach and textbooks. This means that they may find the same things difficult to understand and have common difficulties in communicating with domain experts. It is quite possible that they will, independently, make the same mistakes and design the same algorithms to solve a problem.
2. If the requirements are incorrect or they are based on misunderstandings about the environment of the system, then these mistakes will be reflected in each implementation of the system.
3. In a critical system, the V-spec is a detailed document based on the system's requirements, which provides full details to the teams on how the system should

behave. There cannot be scope for interpretation by the software developers. If there are errors in this document, then these will be presented to all of the development teams and implemented in all versions of the system.

One way to reduce the possibility of common specification errors is to develop detailed specifications for the system independently, and to define the specifications in different languages. One development team might work from a formal specification, another from a state-based system model, and a third from a natural language specification. This helps avoid some errors of specification interpretation, but does not get around the problem of specification errors. It also introduces the possibility of errors in the translation of the requirements, leading to inconsistent specifications.

In an analysis of the experiments, Hatton (1997), concluded that a three-channel system was somewhere between five to nine times more reliable than a single-channel system. He concluded that improvements in reliability that could be obtained by devoting more resources to a single version could not match this and so N-version approaches were likely to lead to more reliable systems than single version approaches.

What is unclear, however, is whether the improvements in reliability from a multiversion system are worth the extra development costs. For many systems, the extra costs may not be justifiable as a well-engineered single version system may be good enough. It is only in safety and mission critical systems, where the costs of failure are very high, that multiversion software may be required. Even in such situations (e.g., a spacecraft system), it may be enough to provide a simple backup with limited functionality until the principal system can be repaired and restarted.

13.4 Dependable programming

Generally, I have avoided discussions of programming in this book because it is almost impossible to discuss programming without getting into the details of a specific programming language. There are now so many different approaches and languages used for software development that I have avoided using a single language for examples in this book. However, when considering dependability engineering, there is a set of accepted good programming practices that are fairly universal and which help reduce faults in delivered systems.

A list of good practice guidelines is shown in Figure 13.8. They can be applied in whatever programming language is used for systems development, although the way they are used depends on the specific languages and notations that are used for system development.

Guideline 1: Control the visibility of information in a program

A security principle that is adopted by military organizations is the ‘need to know’ principle. Only those individuals who need to know a particular piece of information in order to carry out their duties are given that information. Information that is not directly relevant to their work is withheld.

Dependable programming guidelines

1. Limit the visibility of information in a program
2. Check all inputs for validity
3. Provide a handler for all exceptions
4. Minimize the use of error-prone constructs
5. Provide restart capabilities
6. Check array bounds
7. Include timeouts when calling external components
8. Name all constants that represent real-world values

Figure 13.8 Good practice guidelines for dependable programming

When programming, you should adopt an analogous principle to control access to the variables and data structures that you use. Program components should only be allowed access to data that they need for their implementation. Other program data should be inaccessible, and hidden from them. If you hide information, it cannot be corrupted by program components that are not supposed to use it. If the interface remains the same, the data representation may be changed without affecting other components in the system.

You can achieve this by implementing data structures in your program as abstract data types. An abstract data type is a data type in which the internal structure and representation of a variable of that type is hidden. The structure and attributes of the type are not externally visible and all access to the data is through operations. For example, you might have an abstract data type that represents a queue of requests for service. Operations should include get and put, which add and remove items from the queue, and an operation that returns the number of items in the queue. You might initially implement the queue as an array but subsequently decide to change the implementation to a linked list. This can be achieved without any changes to code using the queue, because the queue representation is never directly accessed.

You can also use abstract data types to implement checks that an assigned value is within range. For example, say you wish to represent the temperature of a chemical process, where allowed temperatures are within the range 20–200 degrees Celsius. By including a check on the value being assigned within the abstract data type operation, you can ensure that the value of the temperature is never outside the required range.

In some object-oriented languages, you can implement abstract data types using interface definitions, where you declare the interface to an object without reference to its implementation. For example, you can define an interface Queue, which supports methods to place objects onto the queue, remove them from the queue, and query the size of the queue. In the object class that implements this interface, the attributes and methods should be private to that class.

Guideline 2: Check all inputs for validity

All programs take inputs from their environment and process them. The specification makes assumptions about these inputs that reflect their real-world use. For example, it may be assumed that a bank account number is always an eight digit

positive integer. In many cases, however, the system specification does not define what actions should be taken if the input is incorrect. Inevitably, users will make mistakes and will sometimes enter the wrong data. Sometimes, as I discuss in Chapter 14, malicious attacks on a system rely on deliberately entering incorrect input. Even when the input comes from sensors or other systems, these systems can go wrong and provide incorrect values.

You should therefore always check the validity of inputs as soon as these are read from the program's operating environment. The checks involved obviously depend on the inputs themselves but possible checks that may be used are as follows:

1. *Range checks* You may expect inputs to be within a particular range. For example, an input that represents a probability should be within the range 0.0 to 1.0; an input that represents the temperature of a liquid water should be between 0 degrees Celsius and 100 degrees Celsius, and so on.
2. *Size checks* You may expect inputs to be a given number of characters (e.g., eight characters to represent a bank account). In other cases, the size may not be fixed but there may be a realistic upper limit. For example, it is unlikely that a person's name will have more than 40 characters.
3. *Representation checks* You may expect an input to be of a particular type, which is represented in a standard way. For example, people's names do not include numeric characters, e-mail addresses are made up of two parts, separated by a @ sign, etc.
4. *Reasonableness checks* Where an input is one of a series and you know something about the relationships between the members of the series, then you can check that an input value is reasonable. For example, if the input value represents the readings of a household electricity meter, then you would expect the amount of electricity used to be approximately the same as in the corresponding period in the previous year. Of course, there will be variations but order of magnitude differences suggest that a problem has arisen.

The actions that you take if an input validation check fails depend on the type of system being implemented. In some cases, you report the problem to the user and request that the value be reinput. Where a value comes from a sensor, you might use the most recent valid value. In embedded real-time systems, you might have to estimate the value based on history, so that the system can continue in operation.

Guideline 3: Provide a handler for all exceptions

During program execution, errors or unexpected events inevitably occur. These may arise because of a program fault or may be a result of unpredictable external circumstances. An error or an unexpected event that occurs during the execution of a program is called an 'exception'. Examples of exceptions might be a system power failure, an attempt to access non-existent data, or numeric overflow or underflow.

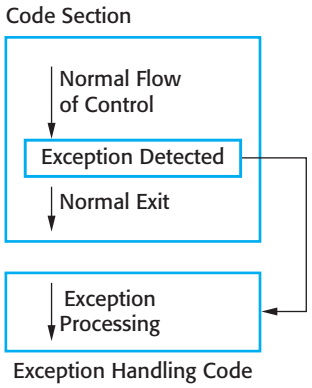


Figure 13.9 Exception handling

Exceptions may be caused by hardware or software conditions. When an exception occurs, it must be managed by the system. This can be done within the program itself or may involve transferring control to a system exception handling mechanism. Typically, the system's exception management mechanism reports the error and shuts down execution. Therefore, to ensure that program exceptions do not cause system failure, you should define an exception handler for all possible exceptions that may arise, and make sure that all exceptions are detected and explicitly handled.

In programming languages such as C, if-statements must be used to detect exceptions and to transfer control to the exception handling code. This means that you have to explicitly check for exceptions wherever in the program they may occur. However, this approach adds significant complexity to the task of exception handling, increasing the chances that you will make mistakes and therefore mishandle the exception.

Some programming languages, such as Java, C++, and Ada, include constructs that support exception handling so that you do not need extra conditional statements to check for exceptions. These programming languages include a special built-in type (often called Exception) and different exceptions may be declared to be of this type. When an exceptional situation occurs, the exception is signaled and the language runtime system transfers control to an exception handler. This is a code section that states exception names and appropriate actions to handle each exception (Figure 13.9). Notice that the exception handler is outside the normal flow of control and that this normal control flow does not resume after the exception has been handled.

Exception handlers usually do one or more of three things:

1. Signal to a higher-level component that an exception has occurred, and provide information to that component about the type of exception. You use this approach when one component calls another and the calling component needs to know if the called component has executed successfully. If not, it is up to the calling component to take action to recover from the problem.

2. Carry out some alternative processing to that which was originally intended. Therefore, the exception handler takes some actions to recover from the problem. Processing may then continue as normal or the exception handler

may indicate that an exception has occurred so that a calling component is aware of the problem.

3. Pass control to a run-time support system that handles the exception. This is often the default when faults occur in a program (e.g., when a numeric value overflows). The usual action of the run-time system is to halt processing. You should only use this approach when it is possible to move the system to a safe and quiescent state, before handing control to the run-time system.

Handling exceptions within a program makes it possible to detect and recover from some input errors and unexpected external events. As such, it provides a degree of fault tolerance—the program detects faults and can take action to recover from these. As most input errors and unexpected external events are usually transient, it is often possible to continue normal operation after the exception has been processed.

Guideline 4: Minimize the use of error-prone constructs

Faults in programs, and therefore many program failures, are usually a consequence of human error. Programmers make mistakes because they lose track of the numerous relationships between the state variables. They write program statements that result in unexpected behavior and system state changes. People will always make mistakes, but in the late 1960s it became clear that some approaches to programming were more likely to introduce errors into a program than others.

Some programming language constructs and programming techniques are inherently error prone and so should be avoided or, at least, used as little as possible. Potentially error-prone constructs include:

1. *Unconditional branch (go-to) statements* The dangers of go-to statements were recognized as long ago as 1968 (Dijkstra, 1968) and, as a consequence, these have been excluded from modern programming languages. However, they are still allowed in languages such as C. The use of go-to statements leads to ‘spaghetti code’ that is tangled and difficult to understand and debug.

2. *Floating-point numbers* The representation of floating-point numbers in a fixed-length memory word is inherently imprecise. This is a particular problem when numbers are compared because representation imprecision may lead to invalid comparisons. For example, 3.00000000 may sometimes be represented as 2.99999999 and sometimes as 3.00000001. A comparison would show these to be unequal. Fixed-point numbers, where a number is represented to a given number of decimal places, are generally safer because exact comparisons are possible.

3. *Pointers* Programming languages such as C and C++ support low-level constructs called pointers, which hold addresses that refer directly to areas of the machine memory (they point to a memory location). Errors in the use of pointers can be devastating if they are set incorrectly and therefore point to the wrong

area of memory. They also make bound checking of arrays and other structures harder to implement.

4. *Dynamic memory allocation* Program memory may be allocated at run-time rather than at compile-time. The danger with this is that the memory may not be properly deallocated, so eventually the system runs out of available memory. This can be a very difficult error to detect because the system may run successfully for a long time before the problem occurs.

5. *Parallelism* When processes are executing concurrently, there can be subtle timing dependencies between them. Timing problems cannot usually be detected by program inspection, and the peculiar combination of circumstances that cause a timing problem may not occur during system testing. Parallelism may be unavoidable, but its use should be carefully controlled to minimize interprocess dependencies.

6. *Recursion* When a procedure or method calls itself or calls another procedure, which then calls the original calling procedure, this is 'recursion'. The use of recursion can result in concise programs; however it can be difficult to follow the logic of recursive programs. Programming errors are therefore more difficult to detect. Recursion errors may result in the allocation of all the system's memory as temporary stack variables are created.

7. *Interrupts* These are a means of forcing control to transfer to a section of code irrespective of the code currently executing. The dangers of this are obvious; the interrupt may cause a critical operation to be terminated.

8. *Inheritance* The problem with inheritance in object-oriented programming is that the code associated with an object is not all in one place. This makes it more difficult to understand the behavior of the object. Hence, it is more likely that programming errors will be missed. Furthermore, inheritance, when combined with dynamic binding, can cause timing problems at run-time. Different instances of a method may be bound to a call, depending on the parameter types. Consequently, different amounts of time will be spent searching for the correct method instance.

9. *Aliasing* This occurs when more than one name is used to refer to the same entity in a program; for example, if two pointers with different names point to the same memory location. It is easy for program readers to miss statements that change the entity when they have several names to consider.

10. *Unbounded arrays* In languages like C, arrays are simply ways of accessing memory and you can make assignments beyond the end of an array. The run-time system does not check that assignments actually refer to elements in the array. Buffer overflow, where an attacker deliberately constructs a program to write memory beyond the end of a buffer that is implemented as an array, is a known security vulnerability.

11. *Default input processing* Some systems provide a default for input processing, irrespective of the input that is presented to the system. This is a security loophole

that an attacker may exploit by presenting the program with unexpected inputs that are not rejected by the system.

Some standards for safety-critical systems development completely prohibit the use of these constructs. However, such an extreme position is not normally practical. All of these constructs and techniques are useful, though they must be used with care. Wherever possible, their potentially dangerous effects should be controlled by using them within abstract data types or objects. These act as natural 'firewalls' limiting the damage caused if errors occur.

Guideline 5: Provide restart capabilities

Many organizational information systems are based around short transactions where processing user inputs takes a relatively short time. These systems are designed so that changes to the system's database are only finalized after all other processing has been successfully completed. If something goes wrong during processing, the database is not updated and so does not become inconsistent. Virtually all e-commerce systems, where you only commit to your purchase on the final screen, work in this way.

User interactions with e-commerce systems usually last a few minutes and involve minimal processing. Database transactions are short, and are usually completed in less than a second. However, other types of systems such as CAD systems and word processing systems involve long transactions. In a long transaction system, the time between starting to use the system and finishing work may be several minutes or hours. If the system fails during a long transaction, then all of the work may be lost. Similarly, in computationally intensive systems such as some e-science systems, minutes or hours of processing may be required to complete the computation. All of this time is lost in the event of a system failure.

In all of these types of systems, you should provide a restart capability that is based on keeping copies of data that is collected or generated during processing. The restart facility should allow the system to restart using these copies, rather than having to start all over from the beginning. These copies are sometimes called checkpoints. For example:

1. In an e-commerce system, you can keep copies of forms filled in by a user and allow them to access and submit these forms without having to fill them in again.
2. In a long transaction or computationally intensive system, you can automatically save data every few minutes and, in the event of a system failure, restart with the most recently saved data. You should also allow for user error and provide a way for users to go back to the most recent checkpoint and start again from there.

If an exception occurs and it is impossible to continue normal operation, you can handle the exception using backward error recovery. This means that you reset the state of the system to the saved state in the checkpoint and restart operation from that point.

Guideline 6: Check array bounds

All programming languages allow the specification of arrays—sequential data structures that are accessed using a numeric index. These arrays are usually laid out in contiguous areas within the working memory of a program. Arrays are specified to be of a particular size, which reflects how they are used. For example, if you wish to represent the ages of up to 10,000 people, then you might declare an array with 10,000 locations to hold the age data.

Some programming languages, such as Java, always check that when a value is entered into an array, the index is within that array. So, if an array A is indexed from 0 to 10,000, an attempt to enter values into elements A [-5] or A [12345] will lead to an exception being raised. However, programming languages such as C and C++ do not automatically include array bound checks and simply calculate an offset from the beginning of the array. Therefore, A [12345] would access the word that was 12345 locations from the beginning of the array, irrespective of whether or not this was part of the array.

The reason why these languages do not include automatic array-bound checking is that this introduces an overhead every time the array is accessed. The majority of array accesses are correct so the bound check is mostly unnecessary and increases the execution time of the program. However, the lack of bound checking leads to security vulnerabilities, such as buffer overflow, which I discuss in Chapter 14. More generally, it introduces a vulnerability into the system that can result in system failure. If you are using a language that does not include array-bound checking, you should always include extra code that ensures the array index is within bounds. This is easily accomplished by implementing the array as an abstract data type, as I have discussed in Guideline 1.

Guideline 7: Include timeouts when calling external components

In distributed systems, components of the system execute on different computers and calls are made across the network from component to component. To receive some service, component A may call component B. A waits for B to respond before continuing execution. However, if component B fails to respond for some reason, then component A cannot continue. It simply waits indefinitely for a response. A person who is waiting for a response from the system sees a silent system failure, with no response from the system. They have no alternative but to kill the waiting process and restart the system.

To avoid this, you should always include timeouts when calling external components. A timeout is an automatic assumption that a called component has failed and will not produce a response. You define a time period during which you expect to receive a response from a called component. If you have not received a response in that time, you assume failure and take back control from the called component. You can then attempt to recover from the failure or tell the system user what has happened and allow them to decide what to do.

Guideline 8: Name all constants that represent real-world values

All non-trivial programs include a number of constant values that represent the values of real-world entities. These values are not modified as the program executes. Sometimes, these are absolute constants and never change (e.g., the speed of light) but more often they are values that change relatively slowly over time. For example, a program to calculate personal tax will include constants that are the current tax rates. These change from year to year and so the program must be updated with the new constant values.

You should always include a section in your program in which you name all real-world constant values that are used. When using the constants, you should refer to them by name rather than by their value. This has two advantages as far as dependability is concerned:

1. You are less likely to make mistakes and use the wrong value. It is easy to mistype a number and the system will often be unable to detect a mistake. For example, say a tax rate is 34%. A simple transposition error might lead to this being mistyped as 43%. However, if you mistype a name (such as Standard-tax-rate), this is usually detected by the compiler as an undeclared variable.
2. When a value changes, you do not have to look through the whole program to discover where you have used that value. All you need do is to change the value associated with the constant declaration. The new value is then automatically included everywhere that it is needed.

KEY POINTS

- Dependability in a program can be achieved by avoiding the introduction of faults, by detecting and removing faults before system deployment, and by including fault-tolerance facilities that allow the system to remain operational after a fault has caused a system failure.
- The use of redundancy and diversity in hardware, software processes, and software systems is essential to the development of dependable systems.
- The use of a well-defined, repeatable process is essential if faults in a system are to be minimized. The process should include verification and validation activities at all stages, from requirements definition through to system implementation.
- Dependable system architectures are system architectures that are designed for fault tolerance. There are a number of architectural styles that support fault tolerance including protection systems, self-monitoring architectures, and N-version programming.
- Software diversity is difficult to achieve because it is practically impossible to ensure that each version of the software is truly independent.

- Dependable programming relies on the inclusion of redundancy in a program to check the validity of inputs and the values of program variables.
- Some programming constructs and techniques, such as go-to statements, pointers, recursion, inheritance, and floating-point numbers, are inherently error prone. You should try to avoid these constructs when developing dependable systems.

FURTHER READING

Software Fault Tolerance Techniques and Implementation. A comprehensive discussion of techniques to achieve software fault tolerance and fault-tolerant architectures. The book also covers general issues of software dependability. (L. L. Pullum, Artech House, 2001.)

'Software Reliability Engineering: A Roadmap'. This survey paper by a leading researcher in software reliability summarizes the state of the art in software reliability engineering as well as discussing future research challenges. (M. R. Lyu, *Proc. Future of Software Engineering*, IEEE Computer Society, 2007.) http://dx.doi.org/10.1109/FOSE.2007.24.

EXERCISES

13.1. Give four reasons why it is hardly ever cost effective for companies to ensure that their software is free of faults.

13.2. Explain why it is reasonable to assume that the use of dependable processes will lead to the creation of dependable software.

13.3. Give two examples of diverse, redundant activities that might be incorporated into dependable processes.

13.4. What is the common characteristic of all architectural styles that are geared to supporting software fault tolerance?

13.5. Imagine you are implementing a software-based control system. Suggest circumstances in which it would be appropriate to use a fault-tolerant architecture, and explain why this approach would be required.

13.6. You are responsible for the design of a communications switch that has to provide 24/7 availability, but which is not safety-critical. Giving reasons for your answer, suggest an architectural style that might be used for this system.

13.7. It has been suggested that the control software for a radiation therapy machine, used to treat patients with cancer, should be implemented using N-version programming. Comment on whether or not you think this is a good suggestion.

13.8. Give two reasons why different versions of a system based around software diversity may fail in a similar way.

13.9. Explain why you should explicitly handle all exceptions in a system that is intended to have a high level of availability.

13.10. The use of techniques for the production of safe software, as discussed in this chapter, obviously includes considerable extra costs. What extra costs can be justified if 100 lives would be saved over the 15-year lifetime of a system? Would the same costs be justified if 10 lives were saved? How much is a life worth? Do the earning capabilities of the people affected make a difference to this judgment?

REFERENCES

Avizienis, A. (1985). 'The N-Version Approach to Fault-Tolerant Software'. *IEEE Trans. on Software Eng.*, **SE-11** (12), 1491–501.

Avizienis, A. A. (1995). 'A Methodology of N-Version Programming'. In *Software Fault Tolerance*. Lyu, M. R. (ed.). Chichester: John Wiley & Sons. 23–46.

Boehm, B. (2002). 'Get Ready for Agile Methods, With Care'. *IEEE Computer,* **35** (1), 64–9.

Brilliant, S. S., Knight, J. C. and Leveson, N. G. (1990). 'Analysis of Faults in an N-Version Software Experiment'. *IEEE Trans. On Software Engineering,* **16** (2), 238–47.

Dijkstra, E. W. (1968). 'Goto statement considered harmful'. *Comm. ACM.,* **11** (3), 147–8.

Hatton, L. (1997). 'N-version design versus one good version'. *IEEE Software,* **14** (6), 71–6.

Knight, J. C. and Leveson, N. G. (1986). 'An experimental evaluation of the assumption of independence in multi-version programming'. *IEEE Trans. on Software Engineering.*, **SE-12** (1), 96–109.

Leveson, N. G. (1995). Safeware: *System Safety and Computers. Reading,* Mass.: Addison-Wesley.

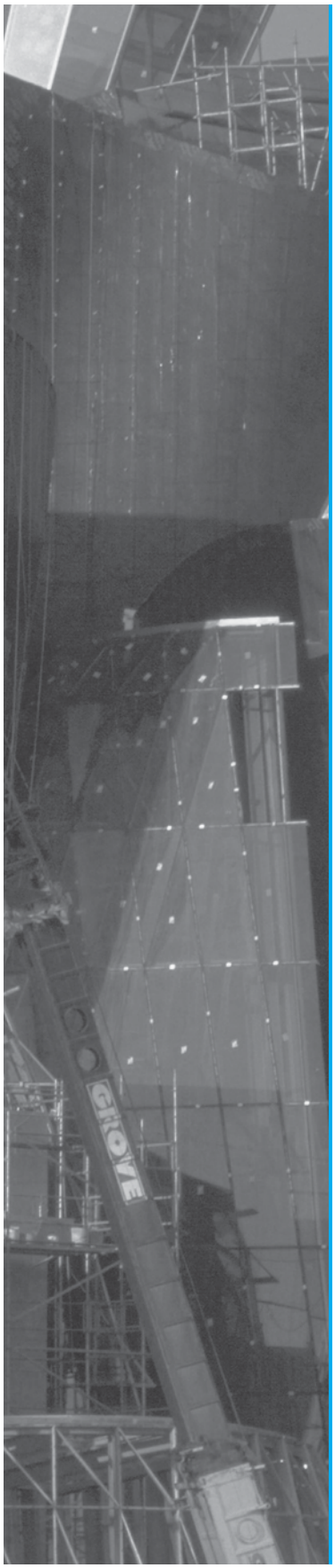
Lindvall, M., Muthig, D., Dagnino, A., Wallin, C., Stupperich, M., Kiefer, D., May, J. and Kahkonen, T. (2004). 'Agile Software Development in Large Organizations'. *IEEE Computer,* **37** (12), 26–34.

Parnas, D. L., Van Schouwen, J. and Shu, P. K. (1990). 'Evaluation of Safety-Critical Software'. *Comm. ACM,* **33** (6), 636–51.

Pullum, L. L. (2001). *Software Fault Tolerance Techniques and Implementation*. Norwood, Mass.: Artech House.

Storey, N. (1996). *Safety-Critical Computer Systems*. Harlow, UK: Addison-Wesley.

Torres-Pomales, W. (2000). 'Software Fault Tolerance: A Tutorial.' http://ntrs.nasa.gov/archive/nasa/casi./20000120144_2000175863.pdf.



14

Security engineering

Objectives

The objective of this chapter is to introduce issues that should be considered when you are designing secure application systems. When you have read this chapter, you will:

- understand the difference between application security and infrastructure security;
- know how life-cycle risk assessment and operational risk assessment are used to understand security issues that affect a system design;
- be aware of software architectures and design guidelines for secure systems development;
- understand the notion of system survivability and why survivability analysis is important for complex software systems.

Contents

14.1 Security risk management
14.2 Design for security
14.3 System survivability

The widespread use of the Internet in the 1990s introduced a new challenge for software engineers—designing and implementing systems that were secure. As more and more systems were connected to the Internet, a variety of different external attacks were devised to threaten these systems. The problems of producing dependable systems were hugely increased. Systems engineers had to consider threats from malicious and technically skilled attackers as well as problems resulting from accidental mistakes in the development process.

It is now essential to design systems to withstand external attacks and to recover from such attacks. Without security precautions, it is almost inevitable that attackers will compromise a networked system. They may misuse the system hardware, steal confidential data, or disrupt the services offered by the system. System security engineering is therefore an increasingly important aspect of the systems engineering process.

Security engineering is concerned with the development and evolution of systems that can resist malicious attacks, which are intended to damage the system or its data. Software security engineering is part of the more general field of computer security. This has become a priority for businesses and individuals as more and more criminals try to exploit networked systems for illegal purposes. Software engineers should be aware of the security threats faced by systems and ways in which these threats can be neutralized.

My intention in this chapter is to introduce security engineering to software engineers, with a focus on design issues that affect application security. The chapter is not about computer security as a whole and so doesn't cover topics such as encryption, access control, authorization mechanisms, viruses and Trojan horses, etc. These are described in detail in general texts on computer security (Anderson, 2008; Bishop, 2005; Pfleeger and Pfleeger, 2007).

This chapter adds to the discussion of security elsewhere in the book. You should read the material here along with:

- Section 10.1, where I explain how security and dependability are closely related;
- Section 10.4, where I introduce security terminology;
- Section 12.1, where I introduce the general notion of risk-driven specification;
- Section 12.4, where I discuss general issues of security requirements specification;
- Section 15.3, where I explain a number of approaches to security testing.

When you consider security issues, you have to consider both the application software (the control system, the information system, etc.) and the infrastructure on which this system is built (Figure 14.1). The infrastructure for complex applications may include:

- an operating system platform, such as Linux or Windows;
- other generic applications that run on that system, such as web browsers and e-mail clients;
- a database management system;

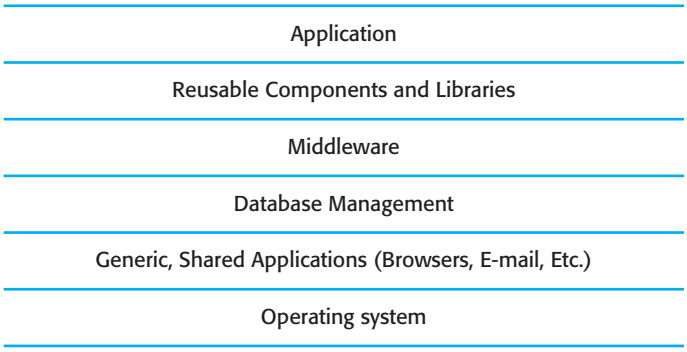


Figure 14.1 System layers where security may be compromised

- middleware that supports distributed computing and database access;
- libraries of reusable components that are used by the application software.

The majority of external attacks focus on system infrastructures because infrastructure components (e.g., web browsers) are well known and widely available. Attackers can probe these systems for weaknesses and share information about vulnerabilities that they have discovered. As many people use the same software, attacks have wide applicability. Infrastructure vulnerabilities may lead to attackers gaining unauthorized access to an application system and its data.

In practice, there is an important distinction between application security and infrastructure security:

1. Application security is a software engineering problem where software engineers should ensure that the system is designed to resist attacks.
2. Infrastructure security is a management problem where system managers configure the infrastructure to resist attacks. System managers have to set up the infrastructure to make the most effective use of whatever infrastructure security features are available. They also have to repair infrastructure security vulnerabilities that come to light as the software is used.

System security management is not a single task but includes a range of activities such as user and permission management, system software deployment and maintenance, and attack monitoring, detection and recovery:

1. User and permission management includes adding and removing users from the system, ensuring that appropriate user authentication mechanisms are in place and setting up the permissions in the system so that users only have access to the resources that they need.
2. System software deployment and maintenance includes installing system software and middleware and configuring these properly so that security vulnerabilities are avoided. It also involves updating this software regularly with new versions or patches, which repair security problems that have been discovered.



Insider attacks and social engineering

Insider attacks are attacks on a system carried out by a trusted individual (an insider) who abuses that trust. For example, a nurse, working in a hospital may access confidential medical records of patients that he or she is not caring for. Insider attacks are difficult to counter because the extra security techniques that may be used would disrupt trustworthy system users.

Social engineering is a way of fooling accredited users into disclosing their credentials. An attacker can therefore behave as an insider when accessing the system.

http://www.SoftwareEngineering-9.com/Web/SecurityEng/insiders.html

3. Attack monitoring, detection and recovery includes activities which monitor the system for unauthorized access, detect, and put in place strategies for resisting attacks, and backup activities so that normal operation can be resumed after an external attack.

Security management is vitally important, but it is not usually considered to be part of application security engineering. Rather, application security engineering is concerned with designing a system so that it is as secure as possible, given budget and usability constraints. Part of this process is 'design for management', where you design systems to minimize the chance of security management errors leading to successful attacks on the system.

For critical control systems and embedded systems, it is normal practice to select an appropriate infrastructure to support the application system. For example, embedded system developers usually choose a real-time operating system that provides the embedded application with the facilities that it needs. Known vulnerabilities and security requirements can be taken into account. This means that an holistic approach can be taken to security engineering. Application security requirements may be implemented through the infrastructure or the application itself.

However, application systems in an organization are usually implemented using the existing infrastructure (operating system, database, etc.). Therefore, the risks of using that infrastructure and its security features must be taken into account as part of the system design process.

14.1 Security risk management

Security risk assessment and management is essential for effective security engineering. Risk management is concerned with assessing the possible losses that might ensue from attacks on assets in the system, and balancing these losses against the

costs of security procedures that may reduce these losses. Credit card companies do this all the time. It is relatively easy to introduce new technology to reduce credit card fraud. However, it is often cheaper for them to compensate users for their losses due to fraud than to buy and deploy fraud-reduction technology. As costs drop and attacks increase, this balance may change. For example, credit card companies are now encoding information on an on-card chip instead of a magnetic strip. This makes card copying much more difficult.

Risk management is a business issue rather than a technical issue so software engineers should not decide what controls should be included in a system. It is up to senior management to decide whether or not to accept the cost of security or the exposure that results from a lack of security procedures. Rather, the role of software engineers is to provide informed technical guidance and judgment on security issues. They are, therefore, essential participants in the risk management process.

As I explained in Chapter 12, a critical input to the risk assessment and management process is the organizational security policy. The organizational security policy applies to all systems and should set out what should and should not be allowed. The security policy sets out conditions that should always be maintained by a security system and so helps to identify risks and threats that might arise. The security policy therefore defines what is and what is not allowed. In the security engineering process, you design the mechanisms to implement this policy.

Risk assessment starts before the decision to acquire the system has been made and should continue throughout the system development process and after the system has gone into use (Alberts and Dorofee, 2002). I also introduced, in Chapter 12, the idea that this risk assessment is a staged process:

1. *Preliminary risk assessment* At this stage, decisions on the detailed system requirements, the system design, or the implementation technology have not been made. The aim of this assessment process is to decide if an adequate level of security can be achieved at a reasonable cost. If this is the case, you can then derive specific security requirements for the system. You do not have information about potential vulnerabilities in the system or the controls that are included in reused system components or middleware.

2. *Life-cycle risk assessment* This risk assessment takes place during the system development life cycle and is informed by the technical system design and implementation decisions. The results of the assessment may lead to changes to the security requirements and the addition of new requirements. Known and potential vulnerabilities are identified and this knowledge is used to inform decision making about the system functionality and how it is to be implemented, tested, and deployed.

3. *Operational risk assessment* After a system has been deployed and put into use, risk assessment should continue to take account of how the system is

used and proposals for new and changed requirements. Assumptions about the operating requirement made when the system was specified may be incorrect. Organizational changes may mean that the system is used in different ways from those originally planned. Operational risk assessment therefore leads to new security requirements that have to be implemented as the system evolves.

Preliminary risk assessment focuses on deriving security requirements. In Chapter 12, I show how an initial set of security requirements may be derived from a preliminary risk assessment. In this section, I concentrate on life cycle and operational risk assessment to illustrate how the specification and design of a system are influenced by technology and the way that the system is used.

To carry out a risk assessment, you need to identify the possible threats to a system. One way to do this, is to develop a set of 'misuse cases' (Alexander, 2003; Sindre and Opdahl, 2005). I have already discussed how use cases—typical interactions with a system—may be used to derive system requirements. Misuse cases are scenarios that represent malicious interactions with a system. You can use these to discuss and identify possible threats and, therefore also determine the system's security requirements. They can be used alongside use cases when deriving the system requirements.

Pfleeger and Pfleeger (2007) characterize threats under four headings, which may be used as a starting point for identifying possible misuse cases. These headings are as follows:

1. Interception threats that allow an attacker to gain access to an asset. So, a possible misuse case for the MHC-PMS might be a situation where an attacker gains access to the records of an individual celebrity patient.
2. Interruption threats that allow an attacker to make part of the system unavailable. Therefore, a possible misuse case might be a denial of service attack on a system database server.
3. Modification threats that allow an attacker to tamper with a system asset. In the MHC-PMS, this could be represented by a misuse case where an attacker changes the information in a patient record.
4. Fabrication threats that allow an attacker to insert false information into a system. This is perhaps not a credible threat in the MHC-PMS but would certainly be a threat in a banking system, where false transactions might be added to the system that transfer money to the perpetrator's bank account.

Misuse cases are not just useful in preliminary risk assessment but may be used for security analysis in life-cycle risk analysis and operational risk analysis. They provide a useful basis for playing out hypothetical attacks on the system and assessing the security implications of design decisions that have been made.

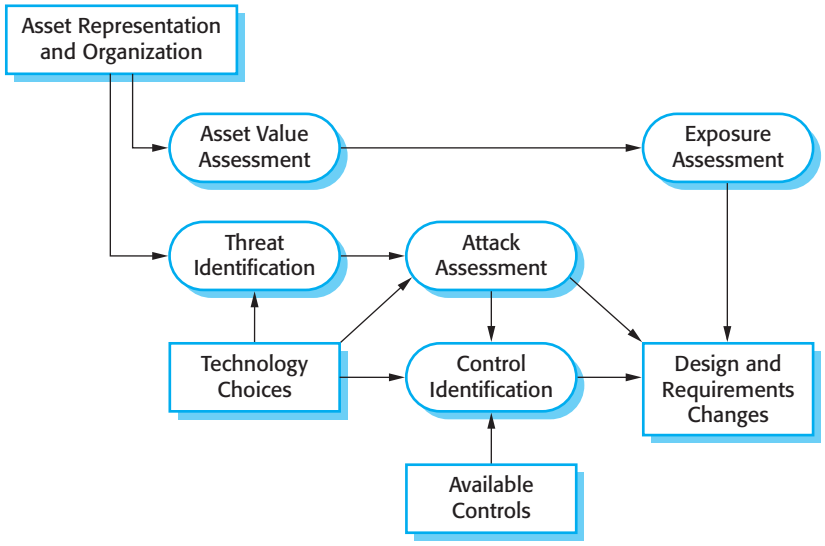


Figure 14.2 Life-cycle risk analysis

14.1.1 Life-cycle risk assessment

Based on organizational security policies, preliminary risk assessment should identify the most important security requirements for a system. These reflect how the security policy should be implemented in that application, identify the assets to be protected, and decide what approach should be used to provide that protection. However, maintaining security is about paying attention to detail. It is impossible for the initial security requirements to take all details that affect security into account.

Life-cycle risk assessment identifies the design and implementation details that affect security. This is the important distinction between life-cycle risk assessment and preliminary risk assessment. Life-cycle risk assessment affects the interpretation of existing security requirements, generates new requirements, and influences the overall design of the system.

When assessing risks at this stage, you should have much more detailed information about what needs to be protected, and you also will know something about the vulnerabilities in the system. Some of these vulnerabilities will be inherent in the design choices made. For example, a vulnerability in all password-based systems is that an authorized user reveals their password to an unauthorized user. Alternatively, if an organization has a policy of developing software in C, you will know that the application may have vulnerabilities because the language does not include array bound checking.

Security risk assessment should be part of all life-cycle activities from requirements engineering to system deployment. The process followed is similar to the preliminary risk assessment process with the addition of activities concerned with design vulnerability identification and assessment. The outcome of the risk assessment is a set of engineering decisions that affect the system design or implementation, or limit the way in which it is used.

A model of the life-cycle risk analysis process, based on the preliminary risk analysis process that I described in Figure 12.9, is shown in Figure 14.2. The most

important difference between these processes is that you now have information about information representation and distribution and the database organization for the high-level assets that have to be protected. You are also aware of important design decisions such as the software to be reused, infrastructure controls and protection, etc. Based on this information, your analysis identifies changes to the security requirements and the system design to provide additional protection for the important system assets.

Two examples illustrate how protection requirements are influenced by decisions on information representation and distribution:

1. You may make a design decision to separate personal patient information and information about treatments received, with a key linking these records. The treatment information is much less sensitive than the personal patient information so may not need as extensive protection. If the key is protected, then an attacker will only be able to access routine information, without being able to link this to an individual patient.

2. Assume that, at the beginning of a session, a design decision is made to copy patient records to a local client system. This allows work to continue if the server is unavailable. It makes it possible for a health-care worker to access patient records from a laptop, even if no network connection is available. However, you now have two sets of records to protect and the client copies are subject to additional risks, such as theft of the laptop computer. You, therefore, have to think about what controls should be used to reduce risk. For example, client records on the laptop may have to be encrypted.

To illustrate how decisions on development technologies influence security, assume that the health-care provider has decided to build a MHC-PMS using an off-the-shelf information system for maintaining patient records. This system has to be configured for each type of clinic in which it is used. This decision has been made because it appears to offer the most extensive functionality for the lowest development cost and fastest deployment time.

When you develop an application by reusing an existing system, you have to accept the design decisions made by the developers of that system. Let us assume that some of these design decisions are as follows:

1. System users are authenticated using a login name/password combination. No other authentication method is supported.

2. The system architecture is client-server, with clients accessing data through a standard web browser on a client PC.

3. Information is presented to users as an editable web form. They can change information in place and upload the revised information to the server.

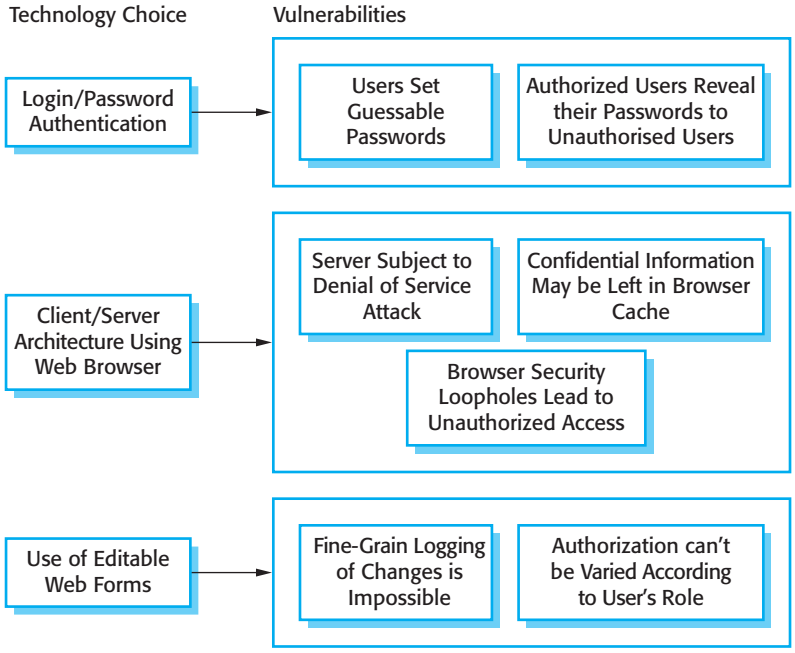


Figure 14.3 Vulnerabilities associated with technology choices

For a generic system, these design decisions are perfectly acceptable, but a life-cycle risk analysis reveals that they have associated vulnerabilities. Examples of possible vulnerabilities are shown in Figure 14.3.

Once vulnerabilities have been identified, you then have to make a decision on what steps that you can take to reduce the associated risks. This will often involve making decisions about additional system security requirements or the operational process of using the system. I don't have space here to discuss all the requirements that might be proposed to address the inherent vulnerabilities, but some examples of requirements might be the following:

1. A password checker program shall be made available and shall be run daily. User passwords that appear in the system dictionary shall be identified and users with weak passwords reported to system administrators.
2. Access to the system shall only be allowed to client computers that have been approved and registered with the system administrators.
3. All client computers shall have a single web browser installed as approved by system administrators.

As an off-the-shelf system is used, it isn't possible to include a password checker in the application system itself, so a separate system must be used. Password checkers analyze the strength of user passwords when they are set up, and notify users if they have chosen weak passwords. Therefore, vulnerable passwords can be identified reasonably

quickly after they have been set up, and action can then be taken to ensure that users change their password.

The second and third requirements mean that all users will always access the system through the same browser. You can decide what is the most secure browser when the system is deployed and install that on all client computers. Security updates are simplified because there is no need to update different browsers when security vulnerabilities are discovered and fixed.

14.1.2 Operational risk assessment

Security risk assessment should continue throughout the lifetime of the system to identify emerging risks and system changes that may be required to cope with these risks. This process is called operational risk assessment. New risks may emerge because of changing system requirements, changes in the system infrastructure, or changes in the environment in which the system is used.

The process of operational risk assessment is similar to the life-cycle risk assessment process, but with the addition of further information about the environment in which the system is used. The environment is important because characteristics of the environment can lead to new risks to the system. For example, say a system is being used in an environment in which users are frequently interrupted. A risk is that the interruption will mean that the user has to leave their computer unattended. It may then be possible for an unauthorized person to gain access to the information in the system. This could then generate a requirement for a password-protected screen saver to be run after a short period of inactivity.

14.2 Design for security

It is generally true that it is very difficult to add security to a system after it has been implemented. Therefore, you need to take security issues into account during the systems design process. In this section, I focus primarily on issues of system design, because this topic isn't given the attention it deserves in computer security books. Implementation issues and mistakes also have a major impact on security but these are often dependent on the specific technology used. I recommend Viega and McGraw's book (2002) as a good introduction to programming for security.

Here, I focus on a number of general, application-independent issues relevant to secure systems design:

1. Architectural design—how do architectural design decisions affect the security of a system?
2. Good practice—what is accepted good practice when designing secure systems?
3. Design for deployment—what support should be designed into systems to avoid the introduction of vulnerabilities when a system is deployed for use?



Denial of service attacks

Denial of service attacks attempt to bring down a networked system by bombarding it with a huge number of service requests. These place a load on the system for which it was not designed and they exclude legitimate requests for system service. Consequently, the system may become unavailable either because it crashes with the heavy load or has to be taken offline by system managers to stop the flow of requests.

http://www.SoftwareEngineering-9.com/Web/Security/DoS.html

Of course, these are not the only design issues that are important for security. Every application is different and security design also has to take into account the purpose, criticality, and operational environment of the application. For example, if you are designing a military system, you need to adopt their security classification model (secret, top secret, etc.). If you are designing a system that maintains personal information, you may have to take into account data protection legislation that places restrictions on how data is managed.

There is a close relationship between dependability and security. The use of redundancy and diversity, which is fundamental for achieving dependability, may mean that a system can resist and recover from attacks that target specific design or implementation characteristics. Mechanisms to support a high level of availability may help the system to recover from so-called denial of service attacks, where the aim of an attacker is to bring down the system and stop it working properly.

Designing a system to be secure inevitably involves compromises. It is certainly possible to design multiple security measures into a system that will reduce the chances of a successful attack. However, security measures often require a lot of additional computation and so affect the overall performance of a system. For example, you can reduce the chances of confidential information being disclosed by encrypting that information. However, this means that users of the information have to wait for it to be decrypted and this may slow down their work.

There are also tensions between security and usability. Security measures sometimes require the user to remember and provide additional information (e.g., multiple passwords). However, sometimes users forget this information, so the additional security means that they can't use the system. Designers therefore have to find a balance between security, performance, and usability. This will depend on the type of system and where it is being used. For example, in a military system, users are familiar with high-security systems and so are willing to accept and follow processes that require frequent checks. In a system for stock trading, however, interruptions of operation for security checks would be completely unacceptable.

14.2.1 Architectural design

As I have discussed in Chapter 11, the choice of software architecture can have profound effects on the emergent properties of a system. If an inappropriate architecture is used, it may be very difficult to maintain the confidentiality and

integrity of information in the system or to guarantee a required level of system availability.

In designing a system architecture that maintains security, you need to consider two fundamental issues:

1. Protection—how should the system be organized so that critical assets can be protected against external attack?
2. Distribution—how should system assets be distributed so that the effects of a successful attack are minimized?

These issues are potentially conflicting. If you put all your assets in one place, then you can build layers of protection around them. As you only have to build a single protection system, you may be able to afford a strong system with several protection layers. However, if that protection fails, then all your assets are compromised. Adding several layers of protection also affects the usability of a system so it may mean that it is more difficult to meet system usability and performance requirements.

On the other hand, if you distribute assets, they are more expensive to protect because protection systems have to be implemented for each copy. Typically, then, you cannot afford as many protection layers. The chances are greater that the protection will be breached. However, if this happens, you don't suffer a total loss. It may be possible to duplicate and distribute information assets so that if one copy is corrupted or inaccessible, then the other copy can be used. However, if the information is confidential, keeping additional copies increases the risk that an intruder will gain access to this information.

For the patient record system, it is appropriate to use a centralized database architecture. To provide protection, you use a layered architecture with the critical protected assets at the lowest level in the system, with various layers of protection around them. Figure 14.4 illustrates this for the patient record system in which the critical assets to be protected are the records of individual patients.

In order to access and modify patient records, an attacker has to penetrate three system layers:

1. *Platform-level protection* The top level controls access to the platform on which the patient record system runs. This usually involves a user signing on to a particular computer. The platform will also normally include support for maintaining the integrity of files on the system, backups, etc.
2. *Application-level protection* The next protection level is built into the application itself. It involves a user accessing the application, being authenticated, and getting authorization to take actions such as viewing or modifying data. Application-specific integrity management support may be available.
3. *Record-level protection* This level is invoked when access to specific records is required, and involves checking that a user is authorized to carry out the requested operations on that record. Protection at this level might also involve

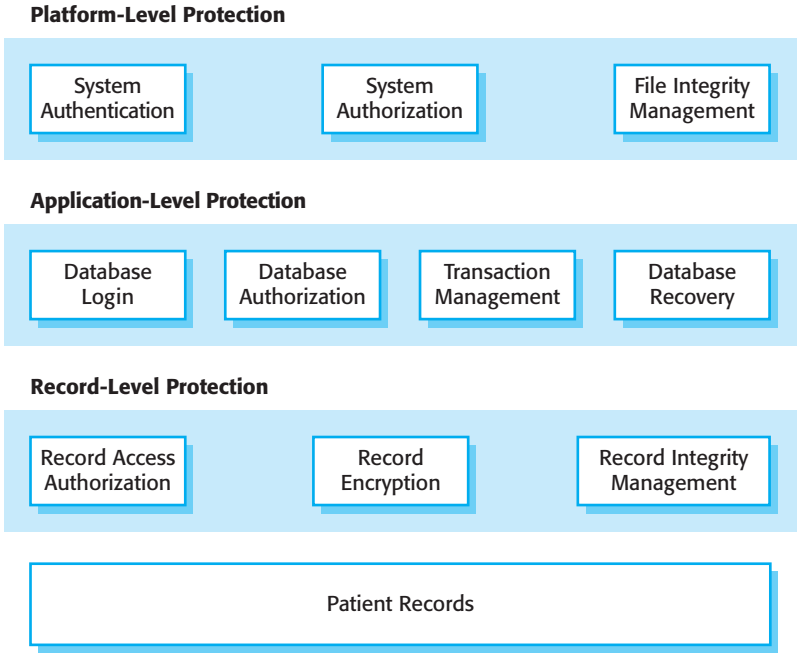


Figure 14.4 A layered protection architecture

encryption to ensure that records cannot be browsed using a file browser. Integrity checking using, for example, cryptographic checksums, can detect changes that have been made outside the normal record update mechanisms.

The number of protection layers that you need in any particular application depends on the criticality of the data. Not all applications need protection at the record level and, therefore, coarser-grain access control is more commonly used. To achieve security, you should not allow the same user credentials to be used at each level. Ideally, if you have a password-based system, then the application password should be different from both the system password and the record-level password. However, multiple passwords are difficult for users to remember and they find repeated requests to authenticate themselves irritating. You often, therefore, have to compromise on security in favor of system usability.

If protection of data is a critical requirement, then a client–server architecture should be used, with the protection mechanisms built into the server. However, if the protection is compromised, then the losses associated with an attack are likely to be high, as are the costs of recovery (e.g., all user credentials may have to be reissued). The system is vulnerable to denial of service attacks, which overload the server and make it impossible for anyone to access the system database.

If you think that denial of service attacks are a major risk, you may decide to use a distributed object architecture for the application. In this situation, illustrated in Figure 14.5, the system's assets are distributed across a number of different platforms, with separate protection mechanisms used for each of these. An attack on one node might mean that some assets are unavailable but it would still be possible to

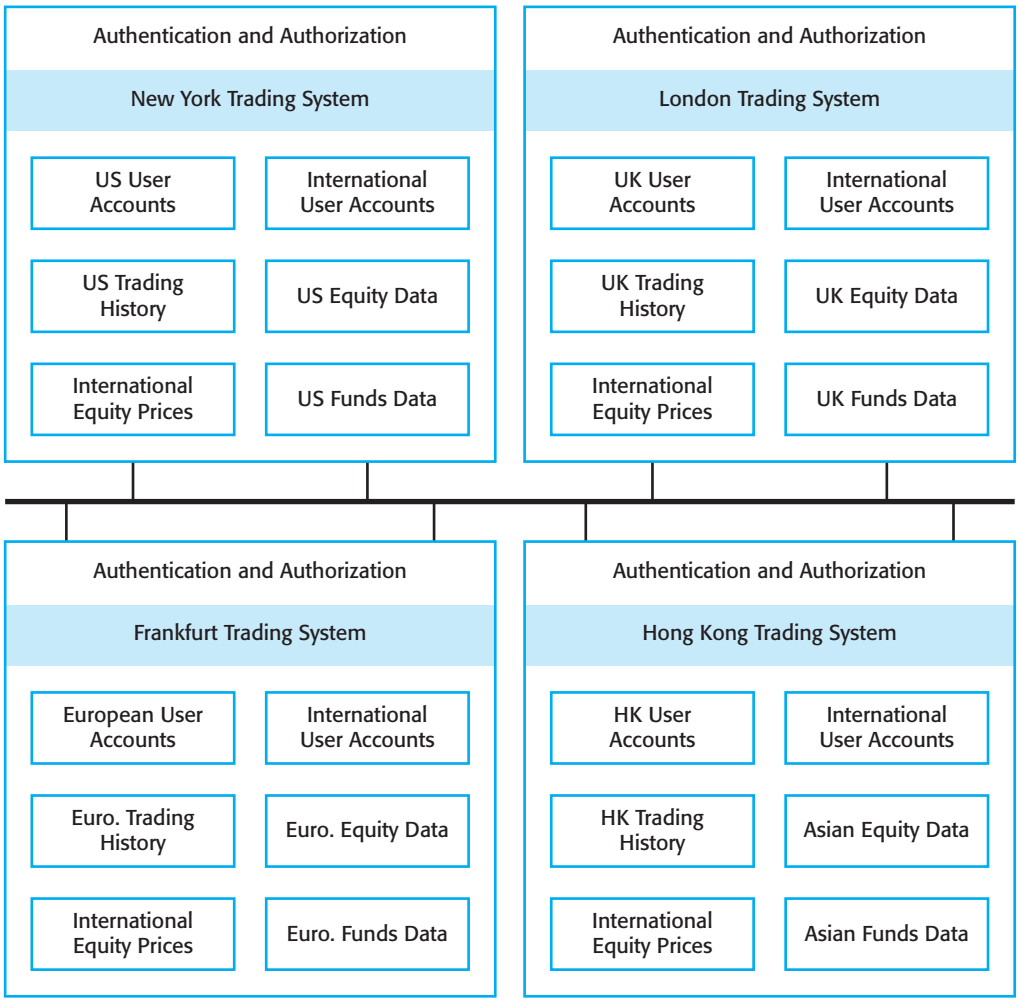


Figure 14.5 Distributed assets in an equity trading system

provide some system services. Data can be replicated across the nodes in the system so that recovery from attacks is simplified.

Figure 14.5 shows the architecture of a banking system for trading in stocks and funds on the New York, London, Frankfurt, and Hong Kong markets. The system is distributed so that data about each market is maintained separately. Assets required to support the critical activity of equity trading (user accounts and prices) are replicated and available on all nodes. If a node of the system is attacked and becomes unavailable, the critical activity of equity trading can be transferred to another country and so can still be available to users.

I have already discussed the problem of finding a balance between security and system performance. A problem of secure system design is that in many cases, the architectural style that is most suitable for meeting the security requirements may not be the best one for meeting the performance requirements. For example, say an

application has one absolute requirement to maintain the confidentiality of a large database and another requirement for very fast access to that data. A high level of protection suggests that layers of protection are required, which means that there must be communications between the system layers. This has an inevitable performance overhead, thus will slow down access to the data. If an alternative architecture is used, then implementing protection and guaranteeing confidentiality may be more difficult and expensive. In such a situation, you have to discuss the inherent conflicts with the system client and agree on how these are to be resolved.

14.2.2 Design guidelines

There are no hard and fast rules about how to achieve system security. Different types of systems require different technical measures to achieve a level of security that is acceptable to the system owner. The attitudes and requirements of different groups of users profoundly affect what is and is not acceptable. For example, in a bank, users are likely to accept a higher level of security, and hence more intrusive security procedures than, say, in a university.

However, there are general guidelines that have wide applicability when designing system security solutions, which encapsulate good design practice for secure systems engineering. General design guidelines for security, such as those discussed, below, have two principal uses:

1. They help raise awareness of security issues in a software engineering team. Software engineers often focus on the short-term goal of getting the software working and delivered to customers. It is easy for them to overlook security issues. Knowledge of these guidelines can mean that security issues are considered when software design decisions are made.
2. They can be used as a review checklist that can be used in the system validation process. From the high-level guidelines discussed here, more specific questions can be derived that explore how security has been engineered into a system.

The 10 design guidelines, summarized in Figure 14.6, have been derived from a range of different sources (Schneier, 2000; Viega and McGraw, 2002; Wheeler, 2003). I have focused here on guidelines that are particularly applicable to the software specification and design processes. More general principles, such as ‘Secure the weakest link in a system’, ‘Keep it simple’, and ‘Avoid security through obscurity’ are also important but are less directly relevant to engineering decision making.

Guideline 1: Base security decisions on an explicit security policy

A security policy is a high-level statement that sets out fundamental security conditions for an organization. It defines the ‘what’ of security rather than the ‘how’, so the policy should not define the mechanisms to be used to provide and enforce security. In principle, all aspects of the security policy should be reflected in the system

Security guidelines
1 Base security decisions on an explicit security policy
2 Avoid a single point of failure
3 Fail securely
4 Balance security and usability
5 Log user actions
6 Use redundancy and diversity to reduce risk
7 Validate all inputs
8 Compartmentalize your assets
9 Design for deployment
10 Design for recoverability

Figure 14.6 Design guidelines for secure systems engineering

requirements. In practice, especially if a rapid application development process is used, this is unlikely to happen. Designers, therefore, should consult the security policy as it provides a framework for making and evaluating design decisions.

For example, say you are designing an access control system for the MHC-PMS. The hospital security policy may state that only accredited clinical staff may modify electronic patient records. Your system therefore has to include mechanisms that check the accreditation of anyone attempting to modify the system and that reject modifications from people who are not accredited.

The problem that you may face is that many organizations do not have an explicit systems security policy. Over time, changes may have been made to systems in response to identified problems, but with no overarching policy document to guide the evolution of a system. In such situations, you need to work out and document the policy from examples, and confirm it with managers in the company.

Guideline 2: Avoid a single point of failure

In any critical system, it is good design practice to try to avoid a single point of failure. This means that a single failure in part of the system should not result in an overall systems failure. In security terms, this means that you should not rely on a single mechanism to ensure security, rather you should employ several different techniques. This is sometimes called ‘defense in depth’.

For example, if you use a password to authenticate users to a system, you might also include a challenge/response authentication mechanism where users have to pre-register questions and answers with the system. After password authentication, they must then answer questions correctly before being allowed access. To protect

the integrity of data in a system, you might keep an executable log of all changes made to the data (see Guideline 5). In the event of a failure, you can replay the log to re-create the data set. You might also make a copy of all data that is modified before the change is made.

Guideline 3: Fail securely

System failures are inevitable in all systems and, in the same way that safety-critical systems should always fail-safe, security critical systems should always 'fail-secure'. When the system fails, you should not use fallback procedures that are less secure than the system itself. Nor should system failure mean that an attacker can access data that would not normally be allowed.

For example, in the patient information system, I suggested a requirement that patient data should be downloaded to a system client at the beginning of a clinic session. This speeds up access and means that access is possible if the server is unavailable. Normally, the server deletes this data at the end of the clinic session. However, if the server has failed, then there is the possibility that the information will be maintained on the client. A fail-secure approach in those circumstances is to encrypt all patient data stored on the client. This means that an unauthorized user cannot read the data.

Guideline 4: Balance security and usability

The demands of security and usability are often contradictory. To make a system secure, you have to introduce checks that users are authorized to use the system and that they are acting in accordance with security policies. All of these inevitably make demands on users—they may have to remember login names and passwords, only use the system from certain computers, and so on. These mean that it takes users more time to get started with the system and use it effectively. As you add security features to a system, it is inevitable that it will become less usable. I recommend Cranor and Garfinkel's book (2005) that discusses a wide range of issues in the general area of security and usability.

There comes a point where it is counterproductive to keep adding on new security features at the expense of usability. For example, if you require users to input multiple passwords or to change their passwords to impossible-to-remember character strings at frequent intervals, they will simply write down these passwords. An attacker (especially an insider) may then be able to find the passwords that have been written down and gain access to the system.

Guideline 5: Log user actions

If it is practically possible to do so, you should always maintain a log of user actions. This log should, at least, record who did what, the assets used, and the time and date of the action. As I discuss in Guideline 2, if you maintain this as a list of executable commands, you have the option of replaying the log to recover from failures. Of course, you also need tools that allow you to analyze the log and detect potentially anomalous actions. These tools can scan the log and find anomalous actions, and thus help detect attacks and trace how the attacker gained access to the system.

Apart from helping recover from failure, a log of user actions is useful because it acts as a deterrent to insider attacks. If people know that their actions are being logged, then they are less likely to do unauthorized things. This is most effective for casual attacks, such as a nurse looking up patient records, or for detecting attacks where legitimate user credentials have been stolen through social engineering. Of course, this is not foolproof, as technically skilled insiders can also access and change the log.

Guideline 6: Use redundancy and diversity to reduce risk

Redundancy means that you maintain more than one version of software or data in a system. Diversity, when applied to software, means that the different versions should not rely on the same platform or be implemented using the same technologies. Therefore, a platform or technology vulnerability will not affect all versions and so lead to a common failure. I explained in Chapter 13 how redundancy and diversity are the fundamental mechanisms used in dependability engineering.

I have already discussed examples of redundancy—maintaining patient information on both the server and the client, firstly in the mental health-care system, and then in the distributed equity trading system shown in Figure 14.5. In the patient records system, you could use diverse operating systems on the client and the server (e.g., Linux on the server, Windows on the client). This ensures that an attack based on an operating system vulnerability will not affect both the server and the client. Of course, you have to trade off such benefits against the increased management cost of maintaining different operating systems in an organization.

Guideline 7: Validate all inputs

A common attack on a system involves providing the system with unexpected inputs that cause it to behave in an unanticipated way. These may simply cause a system crash, resulting in a loss of service, or the inputs could be made up of malicious code that is executed by the system. Buffer overflow vulnerabilities, first demonstrated in the Internet worm (Spafford, 1989) and commonly used by attackers (Berghel, 2001), may be triggered using long input strings. So-called ‘SQL poisoning’, where a malicious user inputs an SQL fragment that is interpreted by a server, is another fairly common attack.

As I explained in Chapter 13, you can avoid many of these problems if you design input validation into your system. Essentially, you should never accept any input without applying some checks to it. As part of the requirements, you should define the checks that should be applied. You should use knowledge of the input to define these checks. For example, if a surname is to be input, you might check that there are no embedded spaces and that the only punctuation used is a hyphen. You might also check the number of characters input and reject inputs that are obviously too long. For example, no one has a family name with more than 40 characters and no addresses are more than 100 characters long. If you use menus to present allowed inputs, you avoid some of the problems of input validation.